

Ppt 1 y 2: Introducción a la ingeniería de sw + ciclo de vida

1. ¿Qué es el software?

El software es una entidad compleja debido a su naturaleza intangible, es conocimiento, información, el resultado de cada actividad del desarrollo de software que genera un artefacto es software.

El software es:

- El código ejecutable (mayor nivel de detalle del conocimiento del software)
- La documentación para desarrollar y mantener los programas ejecutables
- Las herramientas que se usan para hacer el código
- etc

2. Tipos básicos de software

System Software (SOP)	Se encarga de controlar y gestionar los recursos de hardware de una computadora y proporciona una interfaz para que interactúen los usuarios.
Utilitarios	Es un subconjunto del software de sistema. Proporciona herramientas adicionales para mejorar la funcionalidad del sistema operativo, optimizando su rendimiento y el de la computadora.
Software de aplicación	Se usa para realizar tareas específicas en una computadora. Se enfoca en satisfacer las necesidades de los usuarios.

Software ≠ Manufactura

El software NO es comparable con una línea de producción o manufactura porque:

- El software **es menos predecible**, no como los productos tangibles.
- Casi **ningún producto de software es igual a otro**, ya que los requerimientos o necesidades de los usuarios que van a hacer uso de ese software van a tener algunas diferencias.
- **No todas las fallas en software son errores.**
- **El software no se gasta, se adapta.** Cuando hablamos de productos, normalmente, hablamos del tiempo que pasará para que este se desgaste. Cuando hablamos de software hablamos de que el mismo debe ir adaptándose a los cambios y las necesidades del usuario/organización, no tiene la cualidad de desgastarse.
- **El software (obviamente) no está gobernado por las leyes de la física.**

Todas estas diferencias están ligadas a la **intangibilidad del software** como producto

Problemas al desarrollar o construir software

- Se exceden los **tiempos y costos presupuestados**
- La **versión final no satisface al cliente**
- Hay problemas en la **escalabilidad** del software, por lo que agregar una nueva funcionalidad en otra versión es casi imposible.
- **Mala documentación**, desactualizada, y que no refleja al software. Eso trae inconsistencias en la gestión y mantenimiento del mismo.
- **Mala calidad de software**, muchas veces el software de calidad se cree relacionado con la cantidad de testing que realizó sobre él, pero esto no es así nemo.

Para evitar esto es fundamental considerar los factores de éxito del desarrollo de software:

3. Como SI hacer y cómo NO hacer software

<u>Software exitoso</u>	<u>Software NO exitoso</u>
El software termina siendo exitoso y redituable cuando: • Planeamiento adecuado (8,2%) • Expectativas realistas (7,7%) • Personal capacitado y competente (7,2%) • Tenemos apoyo de la gerencia (13%) • Involucramos al usuario (15,9%) • Requerimientos correctamente enunciados (9,6%) • Hitos intermedios (7,7%)	El software NO es exitoso cuando: • Ponemos expectativas poco realistas y demasiado optimistas (9,3%) • Cuando no tenemos suficiente apoyo por parte de la gerencia (8,7%). • No tenemos los recursos suficientes (10,6%) • Cuando tenemos requerimientos incompletos (13.1%) • Requerimientos cambiantes (8,1%) • Poco involucramiento del usuario (12.4%)

Estas son **las premisas de las nuevas formas de trabajar**. Como se descubrió que estos eran los factores de éxito dentro del desarrollo de software, se buscó hacer que los mismos queden embebidos dentro del proceso de desarrollo.

Uno de los aspectos claves tiene que ver con que los **requerimientos**, no solo deben quedar claros, sino ir madurando a medida que se avanza el desarrollo del sistema, para esto es vital hacer que **el usuario tenga participación** para esclarecerlos.

También debemos tener en cuenta la **retroalimentación**, es sumamente importante ir midiendo el avance del sistema y contar con información para poder corregir fallas o errores en el sistema.

4. Qué es la ingeniería de software

Es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

La ingeniería de software se apoya en varias realidades, entre ellas:

- En primer lugar, se debe hacer un esfuerzo para **entender el problema** antes de desarrollar una aplicación de software. Es decir, se debe entender el problema y las necesidades del cliente antes de proponer una solución.
- El **diseño** es una actividad crucial en el desarrollo de un software.
- Realizar un correcto diseño del software, trae consigo dos claros beneficios, **calidad de software y facilidad de mantenimiento**.

5. Disciplinas que conforman la ingeniería de software (la materia en contexto)

- **Disciplinas técnicas:** Actividades que aportan al desarrollo de software como **Producto**. Las vimos en DSI
 - Toma de requerimientos
 - Análisis de requerimientos
 - Diseño de software
 - Implementación de software
 - Testing de software
 - Despliegue de un software
- **Disciplinas de gestión:** Actividades a nivel de **Proyecto**. Para estas se utilizan metodologías de gestión de proyectos.
 - Planificación
 - Monitoreo y control (imposible de hacer sin la planificación)De estas actividades se suele encargar el Project Manager (es el responsable de las mismas), pero el que se encarga de hacerlas a nivel de proyecto es el Project Lider.
- **Disciplinas protectoras o de soporte:** son transversales a los **procesos** de software, permiten verificar la integridad y calidad de un producto de software.
 - SCM (Software Configuration Manager)
 - PPQA (Aseguramiento de calidad de proceso y de producto)
 - Métricas

En la materia vamos a hacer foco en las disciplinas **de gestión y las protectoras**

6. SCM - Software Configuration Manager (Gestión de configuración de software)

Es una **disciplina protectora o de soporte**, transversal (de todo el proceso) cuyo propósito es garantizar la **integridad** del producto.

Garantizar la **integridad** del producto implica que este sea un producto de **calidad**, es decir que satisfaga las necesidades y expectativas del cliente de la mejor manera posible, dentro del presupuesto y costo definido.

Un sistema puede entenderse como un conjunto de versiones, cada versión debe **mantenerse y gestionarse** para no perder la trazabilidad de los **cambios** que se incorporan en cada versión. Estos cambios tienen su origen en los cambios del negocio, nuevos requerimientos del cliente, cambios en los productos asociados, en el presupuesto etc. Ante estos cambios hay que mantener la integridad del producto de software y esto se logra con SCM.

Para esto SCM debe:

- Identificar características técnicas y funcionales de ítems de configuración
- Documentar características técnicas y funcionales de ítems de configuración
- Controlar los cambios de esas características identificadas y documentadas
- Registrar y reportar estos cambios.
- Verificar correspondencia con los requerimientos (trazabilidad).

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

Dijimos que la integridad nos sirve para garantizar que el producto tiene la calidad correspondiente. Pero ¿qué es la integridad?

Decimos que **se mantiene la integridad de un producto de software cuando:**

- Satisface las necesidades del usuario
- Permite la rastreabilidad durante todo su ciclo de vida (saber en donde impactará cada cambio en los ítems de configuración)
- Satisface los criterios de RNF
- Cumple con expectativas de costos (tanto para el cliente como para el equipo del proyecto que debe ver un rédito en su actividad).

a) Elementos de la gestión de configuración:

- 1) **Ítems de configuración** (de proyecto, de producto y de proceso): Son los artefactos que forman parte del producto o proyecto, son cualquier aspecto asociado al producto de software en forma de archivo que se deben mantener (requerimientos, diseño, código fuente, documentos, ERS, arquitectura de software, etc.)

Según la naturaleza del ítem pueden durar lo que dure el ciclo de vida del proyecto, el ciclo de vida del producto o solo un sprint.

- a) Versión: Instancia de un ítem de configuración que difiere de otras instancias de este ítem. “Controlar una versión” es controlar la evolución de un único ítem de configuración o de cada ítem por separado.
- b) Variante: Una variante es una versión de un ítem de configuración que evoluciona por separado, representan configuraciones alternativas.

Los ítems de configuración son almacenados en un:

- 2) **Repositorio**: Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Tiene una estructura para mantener el orden y la integridad.

- a) Rama o brunch: Es un subconjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo.
- b) Integración de ramas o merge: Se da cuando se integran dos ramas, fusiona la configuración de los ítems de cada una sus correspondientes versiones.
- c) Repositorios centralizados vs descentralizados: En el primero un servidor contiene todos los archivos y sus versiones, en los segundos cada cliente tiene una copia exacta e igual al servidor.

- 3) **Configuración de software**: Es una “foto” del repositorio con todos los ítems, con su versión en un momento dado.

4) **Línea base**:

- a) Es un conjunto de ítems de configuración que han sido construidos y revisados formalmente, y aceptados como referencia para demostrar que se ha alcanzado cierto nivel de madurez, y sirven como base para desarrollos posteriores. Son ítems CONFIABLES para avanzar el desarrollo a partir de ellos. Es básicamente una configuración de software que ha sido revisada y aprobada.
- b) Sirve para tener un punto de referencia a la última situación estable y para hacer Rollback.
- c) Hay 2 tipos de ítems de configuración:

- i) De especificación: Son las primeras líneas base, no tienen código sino requerimientos y diseño. Corresponde al primer release.
- ii) Operacionales: Contiene una primera versión del producto con código ejecutable.
- d) Se acuerdan **parámetros** para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.
- e) Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el **Comité de Control de Cambios**, en caso de aceptarse cambios también se acuerdan nuevamente los parámetros y se comunican los cambios a todo el equipo, para que tomen la nueva línea base como modelo a seguir.

b) Planificación de la gestión de configuración de software

Este plan se debe confeccionar al inicio de un proyecto, dentro debe contener:

- Reglas de nombrado de los ítems de configuración;
- Herramientas para utilizar en SCM;
- Roles e integrantes del Comité de Control de Cambios;
- Procedimientos formales de cambios;
- Procesos de auditoría;
- Estructura del repositorio;
- SCM para software externo (opcional);
- Cómo se hará el control de cambios;
- Registros que deben mantenerse;
- Tipos de documentos.

c) Actividades de la gestión de configuración:

Estas actividades están planteadas en un **plan de gestión de configuración**, ya que la gestión de configuración también se planifica.

Estas actividades se realizan en las filosofías tradicional y ágil, salvo las auditorías, ya que ser controlado por alguien externo va en contra del agilismo.



1) Identificación de ítems de configuración: Implica una identificación **unívoca para cada ítem**, donde en el equipo se definirán políticas de nombrado y versionado para todos ellos. También, se debe definir la estructura del repositorio, y la ubicación de los ítems de configuración dentro de esa estructura.

Además, implica la definición de los componentes de la línea base.

Esta identificación y documentación permite a los desarrolladores controlar y velar por la **integridad del producto**.

- Ítems de producto: se mantienen mientras el producto exista.
- Ítems de proyecto: tienen un ciclo de vida de proyecto.
- Ítems de iteración: Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este.

2) Control de cambios: Es un procedimiento formal necesario cuando se requiere cambiar uno o más ítems de configuración de la línea base. Este proceso es llevado a cabo por el **comité de control de cambios**, el cual se reúne para autorizar o denegar los cambios sobre la línea base, formado por representantes de todas las áreas de desarrollo.

Etapas

- a) Se recibe la propuesta de cambio sobre una línea base.
- b) El comité de control de cambios realiza un análisis del impacto de dicho cambio.
- c) En caso de que se autorice el cambio se genera una orden de cambio.
- d) Se realiza el cambio.
- e) El comité aprueba la modificación sobre la línea base y marca la marca como nueva línea base
- f) Se notifica a los interesados los cambios sobre la línea base

El control de cambios permite tener una trazabilidad entre los ítems de configuración, y ante un cambio saber cuáles ítems están afectados por este.

3) Auditorías de configuración:

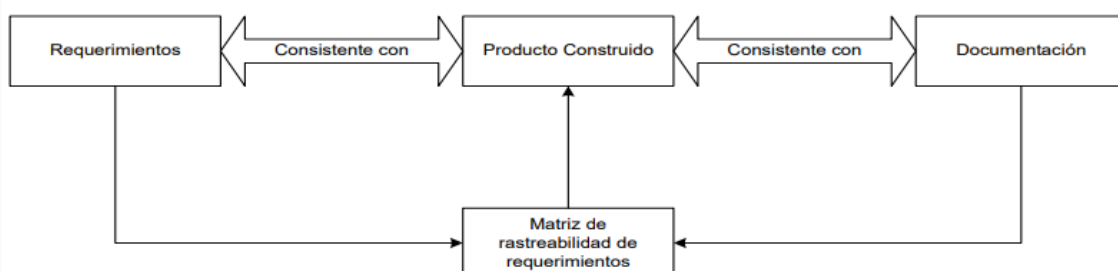
Un auditor **externo** al equipo analiza la línea base la cual le permite evaluar el estado del software en un momento determinado. Esta se hace durante la construcción del producto, para asegurar que este tenga calidad e integridad. Consiste en revisar si se están realizando las tareas como se especificaron en el plan de configuración.

Las auditorías permiten visualizar si se están satisfaciendo los requerimientos y si se ha cumplido la intención de la línea base anterior.

- Primero se hace **la física** (se validan los ítems contra el plan de configuración para asegurar que la documentación es consistente con el código desarrollado)
- Luego se hace la **funcional**: controla que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos, para que el código implemente solo los requerimientos de la ERS.

Auditoría Funcional de Configuración

Auditoría Física de Configuración



Sirve para 2 procesos claves:

- **Validación:** el problema es resuelto de manera apropiada para que el usuario obtenga el producto correcto.
- **Verificación:** asegura que un producto cumple con los objetivos preestablecidos, definidos en la línea base. Todas las funciones son exitosas y los tests pasados o informan los problemas.

4) Reportes e informes de estado:

Es un registro de cómo evoluciona el sistema, y dónde está ubicado el software, comparado con lo que está publicado en la línea base. Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software. El objetivo principal es asegurarse que la información de los cambios llegue a todos los involucrados.

d) Gestión de configuración en Agile:

En las metodologías ágiles la gestión de configuración cambia el enfoque, posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.

Los equipos ágiles son auto organizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles, tampoco hay un comité para el proceso de forma de control de cambios.

SCM en agile:

- Responde a los cambios en lugar de tratar de evitarlos.
- La automatización debe ser aplicada donde sea posible. Esto colabora con la entrega frecuente y rápida de software.
- Eliminar el desperdicio - no agregar nada más que valor.
- Provee retroalimentación continua sobre calidad, estabilidad e integridad.

7. El proceso de software: Hay 2 formas de definirlo

1) Es un conjunto estructurado de actividades que, a raíz de un conjunto de entradas, producen una salida. Las actividades de trabajo del proceso varían según los objetivos y las características del software, en base a esto la organización definirá el proceso de desarrollo.



Los inputs serán requerimientos, las personas, materiales, energía, equipamiento, librerías, hardware, etc.

La salida será un producto o servicio que aporte VALOR.

2) Según CMM: Un proceso de software es un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

- **Procedimientos y métodos:** Definición de cómo se van a llevar a cabo las actividades de desarrollo.
- **Personas:** con habilidades, entrenamiento y motivación.
- **Herramientas y equipos:** que las personas usan, junto con los procedimientos y métodos para producir el producto de software.



8. Tipos de procesos

Tenemos dos filosofías claves que definen dos procesos de desarrollo de software:

- **Procesos definidos**
- **Procesos empíricos**

PROCESOS DEFINIDOS:

Se basan **en el modelo industrial** (inspirados en líneas de producción), según estos procesos tengo un conjunto de **inputs** que van a ser utilizados en un conjunto dado de **actividades**, que van a producir el mismo **resultado** o salida siempre que los inputs sean los mismos.

Asumen que habrá REPETIBILIDAD, o sea que podemos repetir el proceso indefinidamente y obtener los mismos resultados.

Estos procesos son **formalizados**, te dicen PASO a PASO detallando todo lo que necesitas hacer y los roles necesarios para cumplir las tareas.

Se le asigna el rol del proceso a una persona en el proyecto que buscará visibilidad para saber los costos, tiempos, etc.

En caso de que haya que realizar un **cambio en el proceso**, el equipo solo no puede hacerlo, necesitan autorización, y el cambio se hará una vez que haya mucha evidencia que lo justifique.

Ejemplos de procesos definidos: PUD y RUP

Ciclos de vida aceptables para procesos definidos: secuenciales, iterativos y recursivos.

Esto es difícilmente aplicable al software. No podemos simplificar el software a una entrada de inputs que siendo utilizado para determinadas aplicaciones producirán el mismo output, como si el output fuera predecible.

PROCESOS EMPÍRICOS:

Viene del empirismo, es aquella corriente en la cual centramos el foco en la **experiencia**.

Nos basamos en capitalizar **la experiencia de los actores involucrados (el equipo)** en la producción para maximizar la calidad de lo que estamos produciendo. Esta experiencia NO se extrapola, **la experiencia del equipo solo le sirve al equipo**. Por lo tanto tenemos que generar experiencia propia y única del equipo con **frecuencia y pronto** para poder usarla lo antes posible.

En esta filosofía el equipo NO necesita pedir autorización para hacer cambios, este usa su experiencia, por lo tanto es fundamental tener **ciclos de vida cortos y de retroalimentación rápidos (del producto y del proceso)** *1.

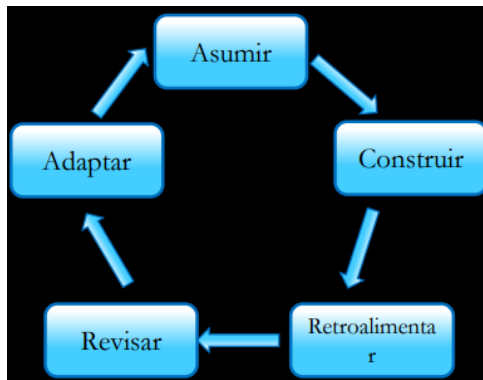
La clave del empirismo son las personas, son quienes capitalizan la experiencia.

Pilares del empirismo:

- **Transparencia:** La información de los productos de trabajo son de acceso para TODOS los miembros del equipo. Busca que se pida ayuda, se admitan errores y que todos puedan generar experiencia y aprender de ella.
- **Inspección:** En un punto de control dado se revisa si los resultados que se están consiguiendo se alinean con lo que se busca.
- **Adaptación:** Se cambia lo que haga falta según lo que se vió en la inspección, SIN requerir autorización.

En cada ciclo inspección - adaptación hay experiencia

Patrón de conocimiento de procesos empíricos



- Empezamos con una hipótesis (ASUMIR)
- Empezamos la etapa de construcción (CONSTRUIR) y en base a algunos puntos que podemos inspeccionar y analizar tomamos mediciones
- sacamos información y haciendo retrospectiva (RETROALIMENTAR)
- revisamos (REVISAR) el resultado de la construcción y contrastando la hipótesis inicial con el resultado del proceso
- podemos ir adaptando el proceso y seguir construyéndolo (ADAPTAR).

Ejemplos de procesos definidos:

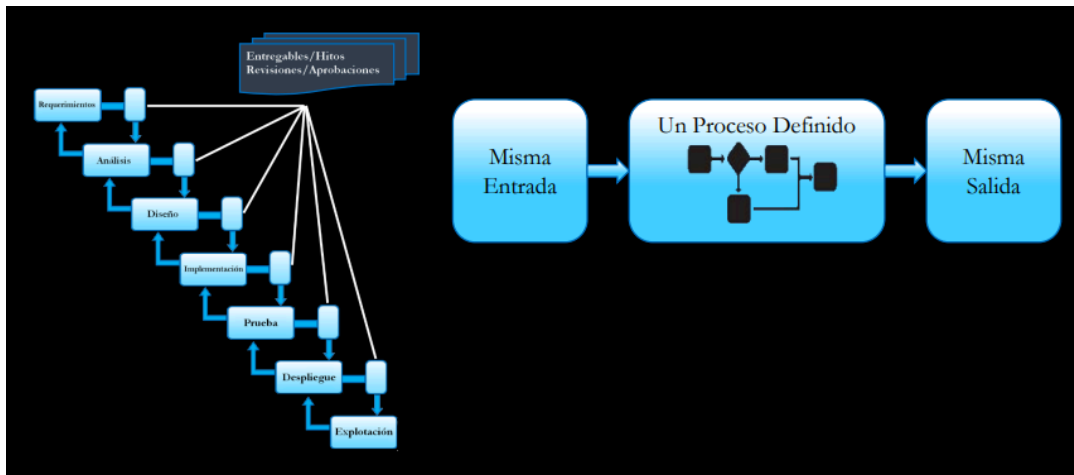
Filosofía ágil: SCRUM
Filosofía lean: Kanban

Ciclos de vida aceptables para procesos definidos: solo iterativos

*1 debido a que se necesita repetición, retroalimentación constante y rápida para adquirir experiencia

Contrastados con los procesos definidos:

- Confiamos en que las variables no pueden ser controladas en su totalidad, por lo tanto tenemos la necesidad de adaptación para poder garantizar la producción de un software de calidad.
- En el proceso definido el punto de mejora del proceso es central (las actividades/etapas intermedias, ya que mis entradas y salidas son fijas), en el proceso empírico no hay un punto de mejora tangible, voy a trabajar sobre distintos aspectos.



9. Ciclo de vida (en general)

El ciclo de vida es una abstracción. Me va a definir **cuáles son las etapas** (fases) y **el orden de cada una de ellas**. No me dice que tengo que hacer, ni quién lo va a hacer, ni las herramientas ni los procedimientos sino **solo las fases (requerimientos, diseño, implementación, etc) y en qué orden se van a ejecutar (según si es secuencial, iterativo o incremental)**. Son la serie de pasos a través de los cuales un producto o un proceso progresa. Existe el ciclo de vida del producto y el ciclo de vida del proyecto, lo cual explico más adelante.

Hay 3 tipos de ciclo de vida básico:

- 1) SECUENCIAL: Ejecutan una etapa después de otra sin retorno generalmente. Se pasa a la siguiente etapa una vez terminada la anterior. Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que éstos se congelan al finalizar la etapa de toma de requerimientos. El trabajo principal es la documentación. Ej: cascada
- 2) ITERATIVO: A diferencia de los secuenciales, el ciclo de vida se divide en iteraciones. Hago iteraciones para ganar información y convertir esa información en un incremento que será incorporado en el sistema. Cada iteración produce un incremento de software funcional potencialmente entregable. Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente.
- 3) RECURSIVO: Se utiliza para casos particulares como proyectos de alto riesgo. Ejemplos: espiral y el ciclo de vida en B.

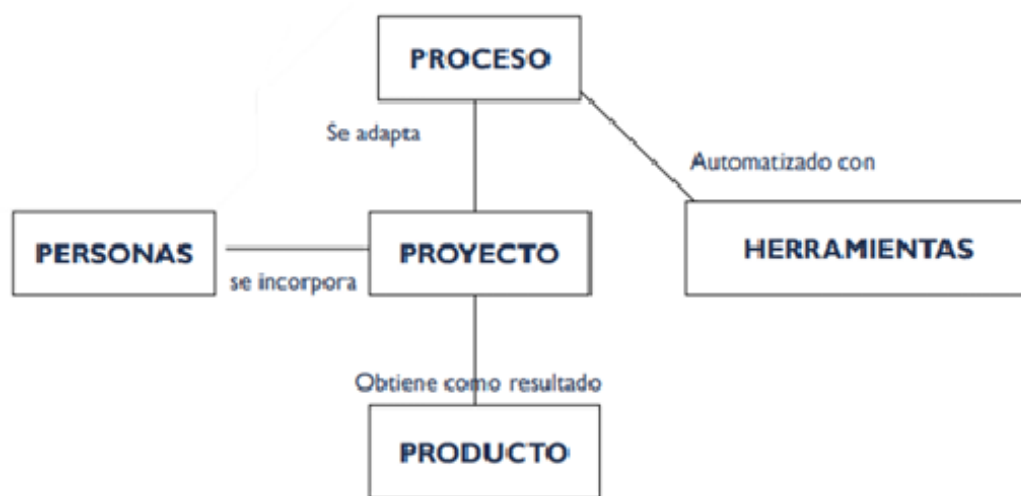
Criterios para la selección de un modelo de proceso

1. Si los requerimientos son claros y exhaustivos, en un contexto de baja incertidumbre en el que el cliente no requiere obtener el producto rápidamente, es posible aplicar el modelo **secuencial**.
2. Si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema, entonces sería conveniente la elección del modelo **iterativo- incremental**.
3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el modelo **recursivo** puede ser una buena opción.

- Los procesos definidos tiene ciclos de vida iterativos-incrementales
- Los procesos empíricos tiene ciclo de vida iterativos-incrementales

Ambos se llaman igual pero son diferentes. El agilismo no creó los ciclos de vida iterativos-incrementales, creo los ciclos de vida iterativos-incrementales **de duración fija**.

10. Vinculo proceso-proyecto-producto



El **PROCESO** es una implementación del **CICLO DE VIDA**. Es un conjunto de actividades estructuradas para hacer el software, es una definición teórica. El proceso proporciona un marco conceptual en donde se establece un plan completo para el desarrollo del software. Que proceso usar es lo primero que se define.

- El **proceso** dice qué actividades llevar a cabo
- y el **ciclo de vida** cómo llevar a cabo esas actividades.

El proceso se **adapta** al **PROYECTO**, este se ayuda de las definiciones puestas en el proceso que adopta para saber qué hacer. Se eligen qué cosas del proceso usar. El proyecto es la unidad de gestión de trabajo, es el medio por el cual se administran los recursos, tareas y personas para obtener el producto. El proyecto hace concretas las actividades del proceso.

El proyecto tiene como **resultado** un **único PRODUCTO**. En realidad una versión del producto. Este es el software instalado y andando, junto con el manual de usuario, las bases de datos, etc, todo el software.

En el proyecto se **integran** las **PERSONAS**. El equipo va a basar su trabajo en el **proceso** adoptado, en el cual se van a tomar sólo aquellas actividades y formas de trabajo que deberán utilizar y seguir para lograr el desarrollo del producto/servicio deseado.

El proceso es **automatizado** con **HERRAMIENTAS**.

PPT 3: Filosofía ágil

1. Manifiesto ágil (principios y valores)

La filosofía ágil NO es una metodología o un proceso, es una ideología con un conjunto definido de **principios y valores**.

Los **4 valores** del agilismo están expresados en el **manifiesto ágil**:



Es fundamental que el cliente esté dispuesto a participar y sumarse a este enfoque, ya que, de lo contrario, la metodología Ágil no se puede aplicar.

Los **12 principios** del manifiesto ágil son:

- 1) **Satisfacción del cliente:** Nuestra mayor prioridad es satisfacer al cliente a través de la entrega temprana y continua de software con valor.
- 2) **Adaptación al cambio:** Aceptamos que los requerimientos cambien, incluso en etapas tardías del desarrollo.
- 3) **Entregas frecuentes:** Entregamos software funcional con frecuencia, antes que documentación exhaustiva.
- 4) **Trabajo en equipo:** Colaboramos con el cliente y los interesados durante todo el proyecto para asegurarnos de que el software entregado satisfaga las necesidades del negocio.
- 5) **Personas motivadas:** Construimos proyectos en torno a individuos motivados.
- 6) **Comunicación directa:** El método más eficiente y efectivo de comunicar información en el equipo es cara a cara.
- 7) **Software funcionando:** El software funcionando es la medida principal de progreso.
- 8) **Continuidad:** El equipo de trabajo debe ser capaz de mantener un ritmo constante y sostenible de trabajo a lo largo del tiempo. Para esto se usan ciclos de tiempo fijos y se entrega lo que se llegue a terminar en ese tiempo.
- 9) **Excelencia técnica:** La calidad del producto no es negociable, debemos ajustar cualquier aspecto del producto menos su calidad al entregar.

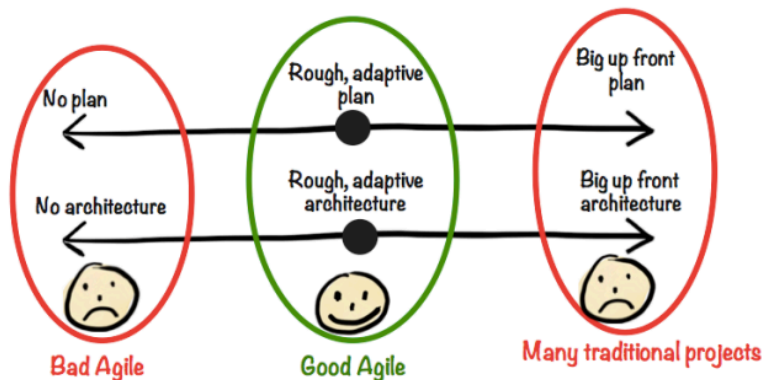
- 10) **Simplicidad:** eliminar tareas innecesarias y concentrarse en los objetivos más importantes y críticos del proyecto para reducir la complejidad y el riesgo. No agregar funcionalidades que el cliente no pidió.
- 11) **Equipos auto-organizados:** Los equipos son auto-gestionados, tienen autonomía y capacidad de decisión. Los más aptos para definir características del productos, son los que van a desarrollarlo.
- 12) **Mejora continua:** El equipo reflexiona sobre cómo ser más efectivo para luego ajustar y perfeccionar su comportamiento en consecuencia durante ciertos intervalos de tiempo.

¿Qué es ser ágil? (y qué no)

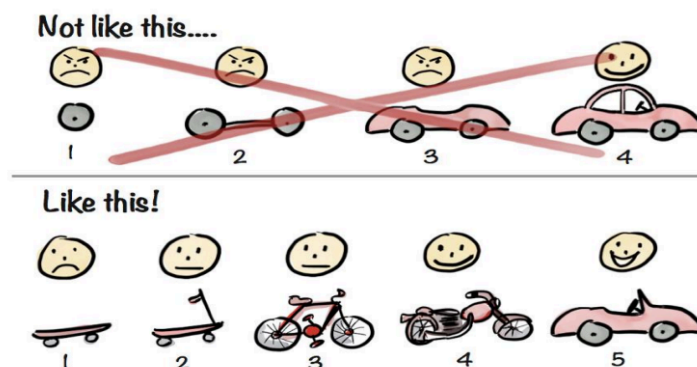
Ser ágil es tener un balance entre ningún proceso y demasiado proceso.

- Los métodos ágiles son adaptables, no predictivos.
- Son orientados a la gente en lugar de al proceso.
- Exigen menos documentación pero sí la exigen.

Ser ágil **NO es ser indisciplinado**, la filosofía ágil define pocas cosas a cumplir, pero son muy estrictos con esas pocas cosas.



Ser ágil **tampoco es hacer las cosas en partes:**



Agilismo y empirismo

El manifiesto ágil se sustenta en los **procesos empíricos** que tienen como base la experiencia del propio equipo, por ello es importante tener ciclos de **realimentación** cortos.

Esto implica mostrarle al cliente software andando lo antes posible para obtener retroalimentación, y a partir de esta, se realizan mejoras y correcciones continuas. Este enfoque es compatible únicamente con **ciclos de vida iterativos**.

Frameworks para procesos empíricos:

Dentro de los procesos empíricos tenemos dos filosofías:

- Ágil: SCRUM, XP, Crystal, FDD, etc.
- Lean: Kanban.

2. Requerimientos ágiles

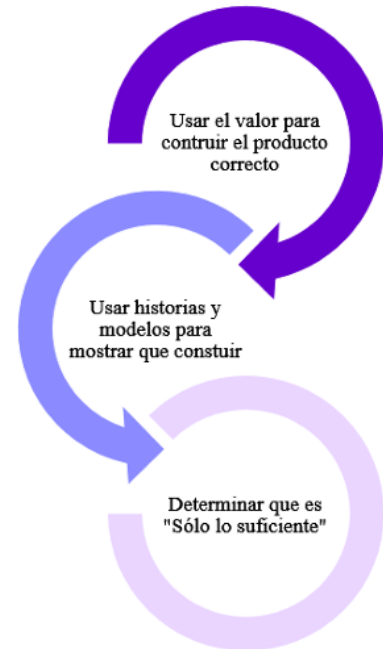
En la gestión ágil los **requerimientos** se van descubriendo a medida que avanza el proyecto, eso para garantizar que **el producto sea el correcto y aporte valor al negocio**.

Para comenzar a trabajar se define lo mínimo necesario y a partir de esto se busca obtener **retroalimentación** lo antes posible.

En lugar de especificar todos los requerimientos desde el inicio, arranco con una versión chica que cubra los requerimientos de alta prioridad (que generan más valor) y se los muestro al cliente para **generar experiencia**.

El fin del agilismo es obtener un OUTCOME o resultado de valor, antes que un output (salida).

En el agilismo los requerimientos se expresan en forma de **historias de usuario**, estas describen los requisitos del cliente y necesidades del usuario en su lenguaje natural. Estas historias se usan para definir el **alcance de cada iteración** del proyecto y se priorizan en función del valor que aportan.

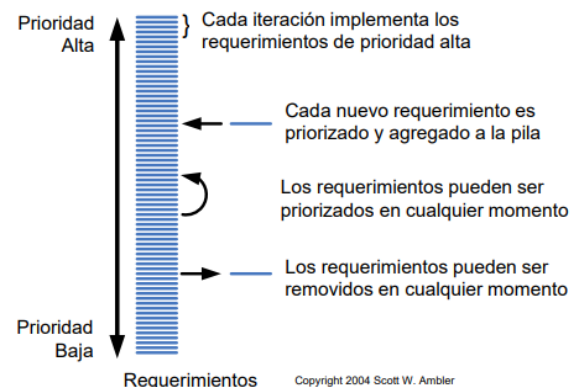


Determinar “**sólo lo suficiente**” nos ayuda a **reducir el desperdicio** que suelen tener los productos de software, con esta filosofía las funciones que se vayan desarrollando serán las que tengan más probabilidades de ser usadas por el cliente.

Para esto el agilismo usa el concepto de **Product Owner**.

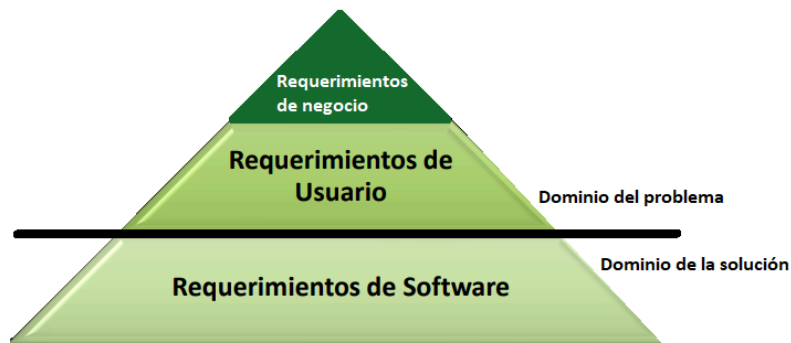
Este es el responsable de identificar las necesidades del negocio y priorizar los requerimientos que aporten un valor más significativo al mismo. Estos requerimientos priorizados se organizan en una lista llamada **Product Backlog**, ordenada de mayor a menor prioridad.

Los requerimientos que están en el tope de la lista son aquellos en los que vamos a trabajar con mayor nivel de detalle. Nunca el Product Backlog tiene TODOS los requerimientos a la vez, porque algunos aún no se identificaron y otros ya se removieron.



Para reducir el desperdicio los requerimientos son analizados solo cuando aportan el suficiente valor, ni antes ni después de ser necesarios. A esto se le llama **Just in time**.

Tipos de requerimientos:



Primero nos enfocamos en determinar los **requerimientos del dominio del problema**.

Al enfocarse en los requerimientos del dominio del problema y diferenciarlos de los requerimientos específicos de software, los equipos de desarrollo pueden tener una visión más clara de lo que el software debe hacer.

En resumen: entendiendo la necesidad del negocio, se descubre la solución de forma colaborativa junto a un equipo motivado y competente para entregar valor a los stakeholders frecuentemente.

Principios ágiles relacionados a los requerimientos ágiles:

- 1. La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes (2 semanas a un mes)
- 2. Recibir cambios de requerimientos, aun en etapas finales.
- 4. Técnicos y no técnicos trabajando juntos todo el proyecto: El PO es parte del equipo.
- 6. El medio de comunicación por excelencia es el cara a cara.
- 11. Las mejores arquitecturas, diseños y requerimientos emergen de equipos auto organizados

3. User stories

Es una técnica para capturar requerimientos. Esta parte del desarrollo de software es la más difícil, ya que los errores cometidos en la etapa de definición de requerimientos son más difíciles de corregir y solo se identifican cuando el usuario tiene el sistema en frente.

A diferencia de los **casos de uso** que detallan lo que debe hacer el sistema, las **user stories** expresan la intención del usuario.

Está planteada como una implementación en **porciones verticales**. Aparte de describir la funcionalidad del usuario, también hay que implementarla desde la lógica de la interfaz de usuario hasta la base de datos.

Story 1	Story 2
GUI	
Business Logic	
Database	

a) Las 3 C de las user stories:

- **Conversación:** Es la parte más importante ya que es cuando se captan las necesidades del problema. En esta conversación (con el PO) el equipo obtiene los detalles necesarios para trabajar en la historia. Es la parte **no visible**. (Principio ágil: el mejor método de comunicación es cara a cara).
- **Card - Tarjeta:** Es la **parte visible** de la user stories, es donde se escribe de qué trata la user story y qué valor aporta al negocio.
- **Confirmation:** Son las condiciones que se tienen que dar para que se dé por satisfecha la US, es lo que el product owner va a mirar para definir si se aprueba o no la user story. Esto también va a la card en forma de **criterios de aceptación y pruebas de usuario**.

b) Las partes de la Card:

- 1) **Frase verbal:** Se usa para hacer referencia a la US, es como un título.
- 2) **Descripción:** COMO [rol] QUIERO [que] PARA [valor de negocio]
El nombre de **rol** representa quién está realizando la acción o quién recibe el valor/beneficio de la actividad/acción; la **actividad** representa la acción que realizará el sistema y el **valor de negocio** que recibo comunica porqué es necesaria la actividad.
 - a) Modelado de roles: Intentamos modelar los roles dentro del sistema para lograr identificar aquellos que probablemente tengamos en las US, y a partir de ellos encontrar en sí las US. Se arma una **Tarjeta de rol de usuario** donde se describe la función del usuario. Tenemos que evitar roles genéricos como “usuario”, “vendedores”, etc.
- 3) **Criterios de aceptación:** Definen los límites de la US, nos dice lo que esta debe hacer desde el punto de vista del **negocio**. Ayudan a que el PO nos describa su intención con la US y **que valor tendría que proveer esta al negocio**. Estos criterios no tienen cuestiones de implementación, sino que se describen desde la conversación. Ayudan a los desarrolladores a saber cuándo dejar de agregar funcionalidad, y a los testers a hacer pruebas pertinentes.
- 4) **Pruebas de usuario:** Nos ayudan a asegurar que el sistema le sirva al usuario. Si todas las pruebas pasan nos confirman que **el sistema hace lo que el PO quiere**. Debemos tener una cantidad de pruebas que deje tranquilo al usuario de que el sistema le va a servir para lo que necesita y nos asegure a nosotros que estamos haciendo las cosas bien, ya que los errores se pagan con la confianza del usuario y esto es caro.
 - Las pruebas de **falla** indican cosas que el sistema no debería permitir.
 - Las pruebas que **pasan** indican cosas que el sistema sí debería permitir.
 - a) No confundir con casos de prueba: Estos examinan la calidad del sistema en pasos detallados.

c) Definition of Ready o Criterio de Listo:

Son los criterios que indican que una User Story está **lista para ser implementada**. Cada equipo elige su definición de listo, así que es diferente para cada uno. Normalmente se arma un checklist y se compara la user contra esta, si cumple con los aspectos acordados puede pasar a ser implementada, sino debe seguir trabajándose.

Modelo INVEST

Para saber si la user está lista se usa el **modelo INVEST**. La US debe ser:

- **I**ndependiente: Que no dependa de otra US, no afecta a otras US moverla dentro del product backlog.
- **N**egociable: Es decir que no es un contrato estricto ni una especificación inamovible. Los detalles de la historia se elaboran en una conversación colaborativa entre el cliente (o Product Owner) y el equipo de desarrollo. Se especifica **QUE** necesita el cliente pero no **CÓMO** cubrir esa necesidad, eso lo decide el equipo técnico.
- **V**alioso: Debe tener un valor concreto para el cliente.
- **E**stimable: Tengo que poder definir cuánto esfuerzo me va a llevar hacer la US.
- **S**mall: La US tiene que entrar en una Sprint (iteración de Scrum). Para saber si es pequeña, tengo que poder estimarla.
- **T**estable: Debe ser posible demostrar de manera objetiva que su funcionalidad ha sido implementada correctamente.

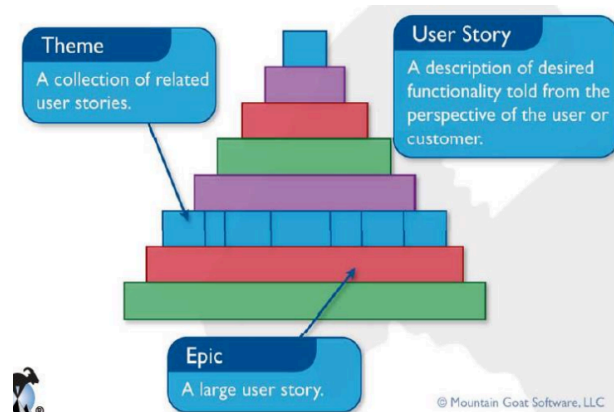
d) Definition of Done:

También son criterios, pero que definen si la US está **lista para ser presentada al PO o al cliente**. Esta definición también es propia del equipo y se valida con una checklist. En caso de pasar la definición de hecho, la US debe pasar las pruebas de usuario y cumplir los criterios de aceptación.

e) Nivel de abstracción:

Según la granularidad con la que se definen los requerimientos podemos tener diferentes niveles de abstracción:

- Las **user stories** son una descripción de una funcionalidad contada desde el punto de vista del usuario. Tiene un alto nivel de detalle.
- Las **épicas** son historias de usuario muy grandes, que quedan al fondo del product backlog hasta que toman prioridad y deban detallarse.
- Los **temas** son un conjunto de US relacionadas que se definen en función de un título, recuerdan que lo que está bajo dicho título se va a tener que tratar en algún momento. Suelen ser más una herramienta de organización.



Respecto al **product backlog**:

1. Una **épica** es un ítem de trabajo concreto pero grande, que se encuentran en el product backlog esperando ser descompuesta en historias más pequeñas a medida que su prioridad aumenta.
2. Los **temas** también pueden estar representados en el Product Backlog, pero a menudo de forma implícita o como una etiqueta para agrupar otros ítems. Son una agrupación de historias y épicas relacionadas, o una iniciativa a muy largo plazo. Son más una herramienta de organización y priorización del product backlog para el

Product Owner que un ítem de trabajo directo para el equipo, el equipo trabaja en las historias y épicas que componen ese tema.

f) Spikes:

Un tipo especial de US son las spikes que sirven para **quitar riesgo o incertidumbre** de otra US o faceta del proyecto.

Por ejemplo: investigar una tecnología que no conocemos, dudas de cómo plantear alguna interacción del usuario con el sistema, analizar una US compleja y dividirla en piezas manejables.

Tenemos 2 tipos de spikes:

- **Spikes técnicos:** son utilizadas para:
 - Investigar enfoques técnicos en el dominio de la solución: evaluar performance potencial, decidir si hacer o comprar, evaluar cierta tecnología, etc.
 - Sirven para cualquier situación en la que el equipo necesite una comprensión más fiable sobre alguna tecnología a aplicar antes de comprometerse a desarrollar una nueva funcionalidad en un tiempo fijo.
- **Spikes funcionales:** son utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema. Suelen ser mejor evaluadas con prototipos.

Normalmente se trabajan siempre en una iteración anterior a la iteración en donde yo decido que voy a implementar la User Story en la que tengo incertidumbre.

Lineamientos para spikes: Las spikes deben ser

- NO se pueden estimar. Se les destina un tiempo arbitrario y si no se llega en ese tiempo se les destina más.
- La excepción y no la regla. Se deben dejar como última opción frente a incógnitas críticas y grandes. Sino el equipo debe aprender a resolver cierta incertidumbre en cada iteración.
- Ser implementada en una iteración separada de las historias resultantes, a menos que sea pequeña y rápida.

En resumen:

- Diferir el análisis detallado tan tarde como sea posible, lo que es justo antes que el trabajo comience. Hasta ese entonces, los requerimientos se capturan en forma de User Stories (como épicas o temas) y se agregan al Product Backlog según el valor que aportan al negocio.
- Las US no son requerimientos de software, no necesitan descripciones exhaustivas de la funcionalidad del sistema

PPt 4 y 7: Estimación

1. ¿Qué es estimar?

La estimación es un proceso que se hace **durante la planificación del proyecto de software**. *Esto lo explico más adelante*

Estimar es predecir en un momento donde no tenemos gran conocimiento, donde hay **incertidumbre**.

Por definición **la estimación no es precisa, siempre tiene un factor de probabilidad**. Cuanto más al inicio del proyecto estamos mayor es la incertidumbre, **a medida que este avanza la incertidumbre va disminuyendo ya que tenemos más información**.

Tenemos que ir ajustando las estimaciones a medida que avanzamos en el desarrollo del proyecto, para garantizar **que se ajuste a la realidad** del desarrollo del software.

Estimar no es planear y planear no es estimar, uno planifica en base a las estimaciones, pero los planes no tienen que ser lo mismo que lo estimado ya que las estimaciones **no son compromisos**. A mayor diferencia entre lo estimado y lo planeado mayor **riesgo**.

2. ¿Para qué estimamos?

Existen dos razones principales por las cuales se llevan adelante las estimaciones:

- Para predecir **completitud**.
- Para administrar **riesgos**.

Para **predecir** el valor de un elemento relacionado con el proyecto o con el producto, por ejemplo, el tiempo que va a llevar, qué costo va a tener el sistema, cuántas personas necesito para realizar el desarrollo, etc.

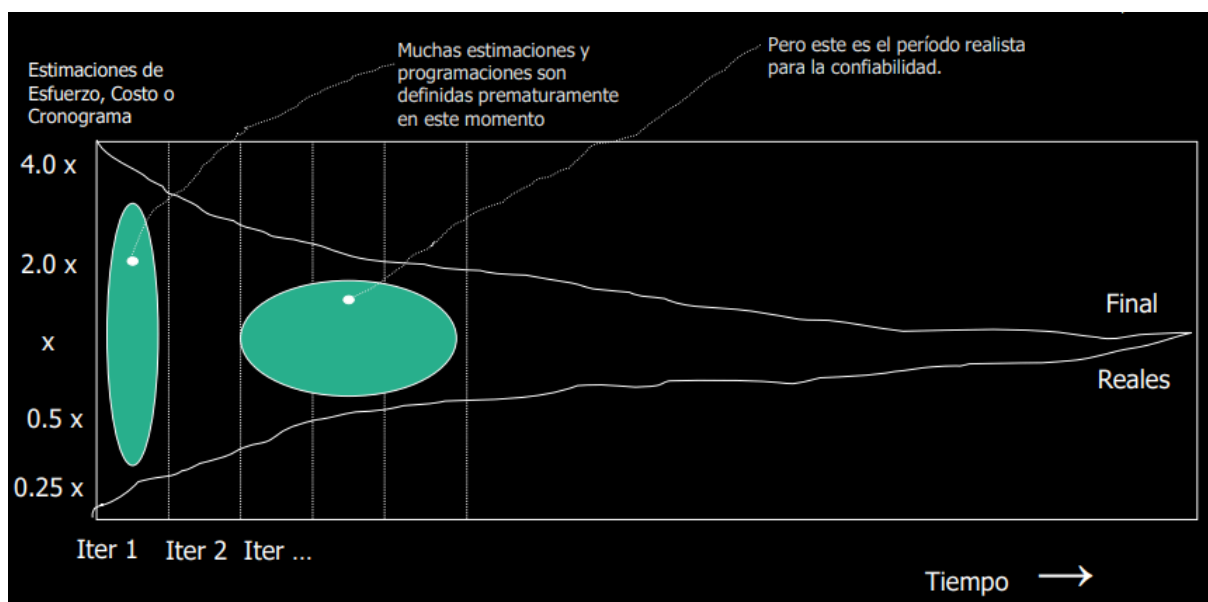
No podemos esperar a tener mucha precisión para hacer estimaciones, sino que tenemos que empezar a trabajar en esas estimaciones **desde un principio**, sabiendo que la incertidumbre es mayor y debemos encontrar la forma de ir disminuyéndola.

Por eso tenemos que estimar sobre algo que tengamos pronto en el proceso de desarrollo.

Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más, esto se debe a:

1. Al inicio del proyecto tenemos poca información y demasiada incertidumbre, lo que aumenta el riesgo de generar estimaciones erróneas.
2. Las estimaciones son de naturaleza optimista, por eso las estimaciones deben ser recalculadas conforme se avanza con el proyecto.
3. Muchas veces estima quien no sabe estimar y planifica quien no sabe planificar.

Hay un periodo una vez entrado el desarrollo que **mejora la confiabilidad** de las estimaciones, ya que entrado el desarrollo disminuye un poco la incertidumbre.



3. Razones de los errores en la estimación

- El **proyecto no está organizado**, por ende el nivel conocimiento sobre el producto es menor y eso hace que las estimaciones sean imprecisas.
- **No tenemos en claro cuáles son los recursos** con los que la empresa cuenta para desarrollar el producto ni las capacidades de los mismos.
- **Las técnicas para hacer estimaciones también pueden ser fuente de error**, se recomienda utilizar más de una técnica y comparar sus resultados. Si tengo resultados muy distintos utilizando diferentes técnicas, tengo errores.

4. ¿Qué y Cómo estimamos (en general)?

- **EL TAMAÑO DEL SOFTWARE:** que tan grande va a ser el trabajo que vamos a hacer. **Contamos:** funcionalidades, requerimientos, cantidad de US, líneas de código, cantidad de CU con su complejidad, etc.
- **EL ESFUERZO:** **Contamos** la cantidad de horas lineales que voy a necesitar para realizar una actividad de la construcción del software. Es decir las horas realmente productivas. Depende del tamaño del proyecto.
- **EL CALENDARIO:** En el calendario ponemos las horas estimadas en el esfuerzo. Son las fechas estimadas. Acá se calendariza el esfuerzo y se proponen fechas.
- **COSTOS Y PRESUPUESTOS:** Es lo último que se estima ya que lo anterior condiciona al costo.

Estas estimaciones varían **según el enfoque que se use en la gestión del proyecto**, con una gestión tradicional se estiman todos estos aspectos, **con una gestión ágil se estima sólo el tamaño(?)**.

5. Métodos y Técnicas de estimación

Tenemos distintas formas o técnicas de estimación que nos ayudan a disminuir/acotar la incertidumbre. Estas técnicas no van a eliminar la incertidumbre, pero nos van a poner en un contexto medianamente esperable.

- **Métodos basados exclusivamente en recursos:** Se basa en la cantidad y el tipo de recursos necesarios para llevar a cabo un proyecto. La estimación se realiza en función de la disponibilidad y costo de los recursos necesarios.
- **Métodos basados exclusivamente en el mercado:** Se basa en el costo de proyectos similares en el mercado para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto.
- **Métodos basados en los componentes del producto o en el proceso de desarrollo:** Se basa en la identificación y descomposición de los componentes del software y del proceso de desarrollo para estimar el esfuerzo y tiempo requerido para cada componente. Luego, se suman las estimaciones para obtener una estimación total del proyecto.
- **Métodos algorítmicos:** Se basa en modelos matemáticos y estadísticos para determinar la estimación de esfuerzo y tiempo requerido para un proyecto.
- **Métodos basados en la experiencia:** Se basa en la experiencia pasada del equipo en proyectos similares para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto.

Dentro de los métodos basados en la experiencia tenemos las siguientes técnicas de estimación:

- **Datos históricos:** Tomo como entrada los datos básicos históricos de un proyecto anterior, para hacer la estimación de mi proyecto, estos son: el tamaño, el esfuerzo, el tiempo y los defectos.
- **Juicio experto:**
 - **Puro:** Un experto estudia las especificaciones y hace su estimación. Se basa fundamentalmente en los conocimientos del experto. Si desaparece el experto, la empresa deja de estimar.
 - **Wideband Delphi:** Un grupo de personas son informadas y tratan de adivinar lo que costará el desarrollo tanto en esfuerzo como en duración. Las estimaciones individuales se recopilan de forma anónima y se envían a los expertos para que puedan revisarlas y ajustarlas en rondas sucesivas. Las estimaciones en grupo suelen ser mejores que las individuales.
Algunos criterios que pueden seguir los expertos para lograr una buena estimación y lograr una estructuración del juicio de experto son:
 - Estimar solo tareas con granularidad aceptable (no muy alta)
 - Estimar según 3 características y luego aplicar la fórmula:
 $(4 * \text{el tiempo habitual} + \text{el tiempo optimista} + \text{el pesimista}) / 6$
 - Usar un checklist y un criterio definido para asegurar cobertura.
- **Analogía:** También se basa en datos históricos pero se seleccionan específicamente aquellos proyectos que son similares al proyecto actual para realizar la comparación y estimación.

6. Estimación tradicional

Cuando planificamos un proyecto con procesos definidos estamos haciendo una gestión tradicional, acá la estimación tiene que ser un **número** y se realizan de la siguiente manera:

En esta gestión quien estima es el líder, o alguien que no va a llevar a cabo el trabajo.

A diferencia del enfoque ágil, en este enfoque la estimación es ABSOLUTA.

El orden de estimación es:

- **Tamaño del PRODUCTO**
- **Esfuerzo**
- **Tiempo/ calendario**
- **Costos**
- **Recursos críticos**

Estimación del tamaño: a nivel de producto

Se toman los requerimientos tomados en la etapa de requisitos. Es una de las más importantes ya que el aumento o disminución del tamaño del producto, afecta de gran manera a otros aspectos,

Una buena métrica para estimar el tamaño del producto es:

Se usan los CU y que tan difíciles son:

CU X Complejidad

Estimación del esfuerzo: a nivel de proyecto

Cuántas horas va a estar una persona enfocada en una actividad por vez.

Esta estimación intenta responder cuántas horas personas lineales serán necesarias para construir el producto, previamente estimado su tamaño.

Horas por personas lineales

Estimación del calendario: a nivel de proyecto

Se calendarizan las horas del esfuerzo. Se divide qué porcentaje de horas se va a dedicar a cada actividad considerando el solapamiento entre las áreas.

Responde a cuándo se van a realizar las tareas definidas. Es necesario realizar esta estimación para poder darle una idea al cliente de **cuánto tiempo se va a demorar la finalización del proyecto**.

Estimación de costos: a nivel de proyecto

Nos basamos en el recurso más significativo, las horas de esfuerzo. Se pueden estimar como horas "planas" (todos los roles cobran lo mismo) o distinguiendo por roles.

Por lo general del presupuesto se destina un

- 80% al esfuerzo
- 10 % a la conectividad
- 10% al resto (licencias, etc)

Estimación de recursos críticos:

Son los recursos que necesito en una cantidad que no tengo, por ejemplo: ancho de banda.

Estimación tradicional y la triple restricción



Las 3 dimensiones sobre las que se puede estimar son **el alcance o requisitos, los recursos o costos y el tiempo**.

En el enfoque tradicional el alcance o requerimientos son fijos y en base a estos se derivan el tiempo y los recursos.

Se aceptan cambios en los requerimientos pero si estos cambian hay que reestimar los tiempos y costos.

7. Estimación ágil

Cuando queremos precisar la estimación en ambientes ágiles seguimos con los mismos conceptos anteriores, pero con algunas diferencias, en estimaciones ágiles puntualmente, se va a trabajar **con las features o stories que** son la unidad mínima de las características de un producto.

El equipo de desarrollo es quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones).

Estimación de tamaño:

En este enfoque las estimaciones NO son absolutas sino RELATIVAS. Lo que significan 8 SP para un equipo de desarrollo, puede no significar lo mismo para otro.

- El **tamaño** de una US se estima con la combinación de su:
 - **Complejidad:** que tan compleja es implementar esa US. Relacionada a la cantidad de partes y relaciones entre las mismas posee la US.
 - **Esfuerzo:** es importante que estime la persona que va a hacer el trabajo, ya que el esfuerzo es distinto entre dos personas. Son las horas persona lineales que se requieren para terminar la US.
 - **Incertidumbre:**
 - Incertidumbre técnica: que es más sobre si se sabe cómo llevar a cabo lo explicitado con respecto a la tecnología a implementar y ese tipo de cosas.
 - Incertidumbre del negocio: si entendemos que es lo que el cliente pide y cómo llevarlo a cabo.

Tamaño vs esfuerzo:

La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación.

Cada user story va a tener su propio **tamaño** o peso, medido en **STORY POINTS** (u otra escala).

Para determinar cuántos Story Points tendrá una US debemos tomar un punto de referencia, este punto de referencia es la **user story canónica**. A esta se le estima una cantidad X de story points para luego compararla con las otras US y de esta forma estimar cuántos story points tendrían las otras stories. Por eso estas estimaciones son **relativas**, porque derivan de la comparación entre la user story canónica y las otras user stories.

Una vez que todas las user stories tienen su respectiva cantidad de story points, **se suman** los SP de todas las user stories y así obtenemos el **tamaño total del producto**.

Una vez estimados los story points de cada user story (el tamaño del producto) hay que estimar el tiempo que va a llevar el proyecto.

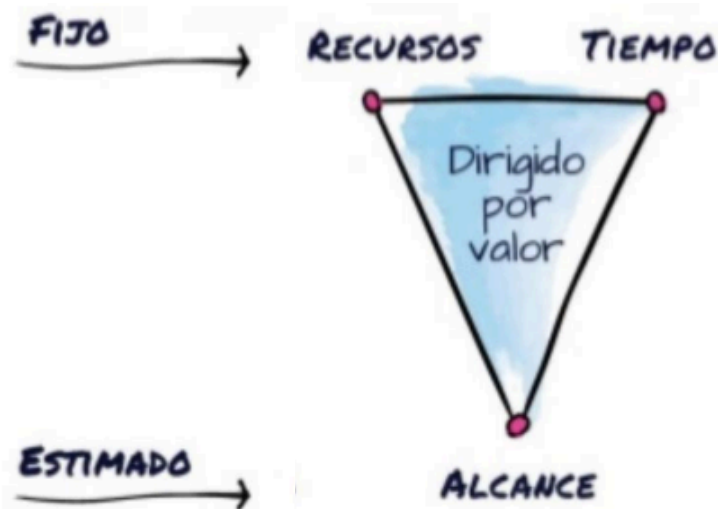
Para esto hay que conocer cuánto va a durar una iteración del proyecto y estimar cuántos story points podremos cubrir en cada iteración.

Una vez que estimas cuántas iteraciones vas a necesitar para cubrir todos los story points sabes cuál es la estimación del tiempo total del proyecto.

Este enfoque es iterativo e incremental, a medida que avance el proyecto se va a ir estimando mejor la cantidad de story points que se llegan a cubrir en cada iteración.

Es por esto que **el tiempo total del proyecto puede ir variando** y no es fijo como en el enfoque tradicional.

Estimación ágil y la triple restricción



En este enfoque los recursos destinados y la duración de cada sprint o iteraciones son fijos.

Lo que varía es el alcance (o la cantidad de story points) que se llegan a cubrir.

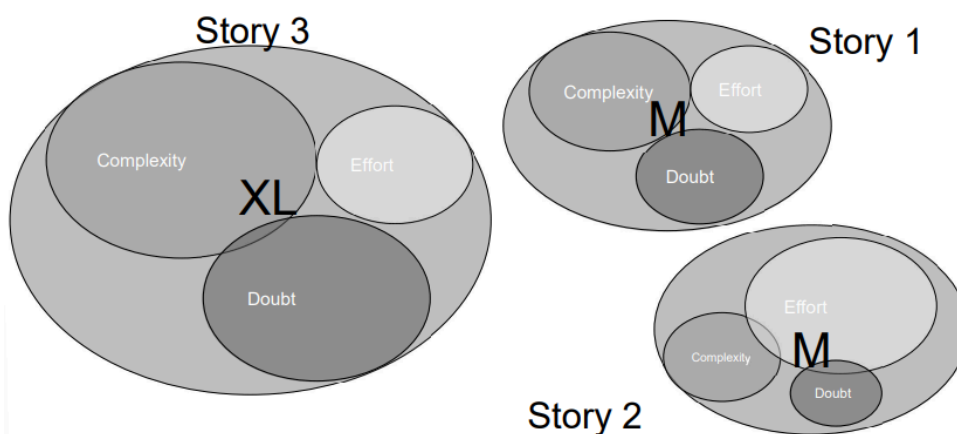
Se pueden introducir cambios en los requerimientos y alcances pero NO DURANTE UN SPRINT.

Se le dice al cliente con este tiempo, y estos recursos ¿Qué querés que hagamos?

Otras escalas de estimación relativas que tenemos son:

- Tamaño por números: 1 a 10
- Talles de remeras: S, M, L, XL, XXL
- Serie 2ⁿ : 1, 2, 4, 8, 16, 32, 64, etc.
- Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

“tamaño” de las Stories



8. Poker planning

Es una técnica de estimación en la que los desarrolladores estiman individualmente cuantos SP le pondrían a una US y luego lo comparten en el grupo. Una vez que todo el equipo ve las estimaciones propuestas aquellos que hayan tenido la estimación más alta y más baja tienen la oportunidad de explicar su razonamiento detrás de los valores elegidos. Luego se hace una nueva ronda de estimación **hasta que haya un consenso en cuanto al valor de Story Points de cada User Story.**

Escala:

Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci como escala para la determinación de los puntos de historia.

1. **Valor 1 o 1/2:** son los valores más bajos y se aplican a una funcionalidad pequeña.

2. **Valor 2- 3:** funcionalidad pequeña a mediana.

3. **Valor 5:** funcionalidad media.

4. **Valor 8:** funcionalidad grande, donde nos preguntamos si es posible dividir.

1. **Valor 13 o más:** necesariamente hay que dividir ya que no ingresa en el sprint.

9. Triángulo de hierro VS Triángulo ágil a nivel de PROYECTO

→ El **triángulo de hierro** es un modelo utilizado en la **gestión tradicional** de proyectos que establece que los tres elementos fundamentales en cualquier proyecto son:

- el alcance
- el tiempo
- y el costo.

En los **proyectos tradicionales** se establece un plan fijo al inicio del proyecto.

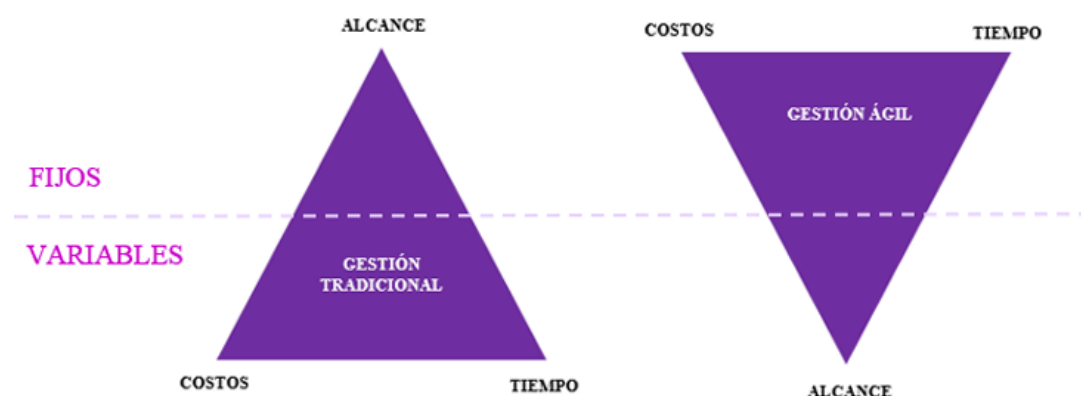
Se negocia el tiempo y los costos y se deja fijo el alcance. El problema es que el alcance siempre está del lado del cliente porque son los requerimientos, osea lo que este necesita que el sistema haga, así que estos siempre pueden cambiar.

El problema es cuando NO se reestiman los costos y tiempo y se toman los cambios en los alcances porque **se sacrifica la calidad**.

En los **proyectos ágiles** utilizan iteraciones cortas y regulares para adaptar el proyecto según las necesidades y los requisitos cambiantes.

Se negocian los alcances y se deja fijos el tiempo y los costos.

Acá **NO se sacrifica la calidad** ya que este modelo, busca **maximizar el valor que se entrega al cliente**.



Triángulo de hierro VS Triángulo ágil usando ambos procesos iterativos

En la gestión tradicional podríamos usar un PUD iterativo incremental. Acá el alcance es FIJO y hasta que no se completa la iteración NO TERMINA

En la gestión ágil podríamos usar SCRUM. Acá la duración del sprint es FIJA y el alcance varía.

PPT 5: Gestión de productos

Un **producto de Software** es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos.

Para hablar de la gestión de productos justamente tenemos que pensar el software como un **producto**, es decir **cambiar la idea de hacer software a medida**.

Con este enfoque primero desarrollamos el producto de software y después buscamos a los usuarios.

¿Por qué crear productos?

El **Leap Time** que es el tiempo que pasa entre que el cliente pide algo y esto le es entregado, es cada vez más corto.

No solo creamos productos para satisfacer a un cliente, ganar dinero y que la aplicación sea popular, sino también para cambiar la visión que se tiene sobre hacer software.

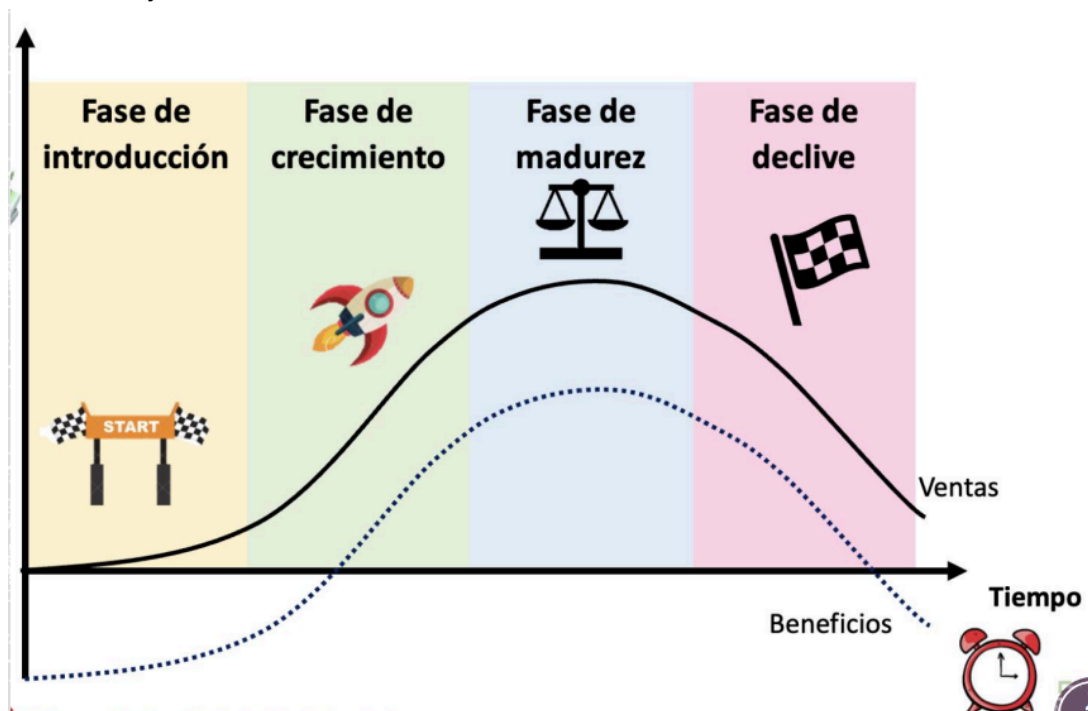
Tipos de productos:

- Tenemos los productos **enlatados** que son productos pre-hechos. El cliente que los encuentra útiles los compra y usa así como vienen.
- Hoy se usa un punto medio entre el software enlatado y el software a medida que es el **producto configurable**. Es decir un producto que ya viene hecho pero que puede ajustarse y configurarse según las preferencias del cliente, sin tocar código.



1. Ciclo de vida del producto

Este ciclo contempla desde la concepción de la idea del producto, hasta que este es discontinuado y se retira del mercado.



- **Fase de introducción:** Es el nacimiento de la idea, se hace un estudio de mercado, si esta fracasa NO se hace el producto
- **Fase de crecimiento**
- **Fase de madurez**
- **Fase de declive:** Cuando se está por vender la compañía

2. Ciclo de vida del producto vs ciclo de vida del proyecto

El ciclo de vida del proyecto lo voy a explicar con más detalle más adelante. Pero la relación entre ambos es que el ciclo de vida del proyecto es más chico que el ciclo de vida del producto. Cada ciclo de vida del proyecto da como resultado **una versión del producto**. El ciclo de vida del producto está formado por varios ciclos de vida de proyecto.

3. Evolución de los productos de software

Cuando vamos a desarrollar un software la mayoría de las veces se empieza a construir un producto sobre la base de lo

- **Funcional**, que el sw haga las tareas que si o si tienen que existir para que el software funcione para el propósito esperado.

quiere avanzar más, el software debe ser

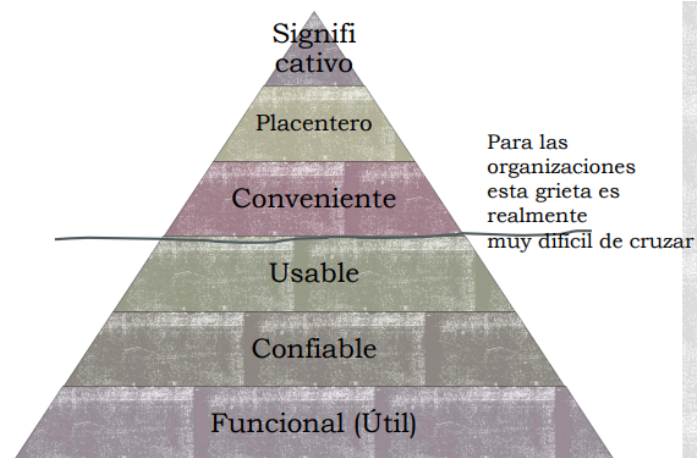
- **Confiable** donde los usuarios no corran peligro usando el producto y los resultados que este producto me da yo pueda depender de ellos sin ningún problema.

Luego, este debe ser

- **Usable**, es decir que el usuario podrá lograr sus objetivos usando este producto, estando a gusto con su uso. Tiene que ser productivo, no es lo mismo que la utilidad. Tiene que ver con aspectos de la disciplina UX/UI. Por ejemplo, si tarda 12 horas en realizar una operación que con el sistema anterior tardaba 4 horas no es productivo y no mejora la calidad de vida al usuario.

Esta primera parte tiene un **foco en las TAREAS** que debe realizar el sistema.

La mayoría de los productos se centran en el **OUTPUT** del sistema y quedan bajo la grieta.



Los productos por encima de la grieta se centran en el **OUTCOME** y tienen un **foco en la EXPERIENCIA DE USUARIO positiva**.

- **Conveniente:** Estos productos nos hacen sentir que lo necesitamos porque nos produce un cambio, una facilidad para cumplir con un objetivo.
- **Placentero y significativo:** son productos que nos cambian la forma de usar algo o de vivir.

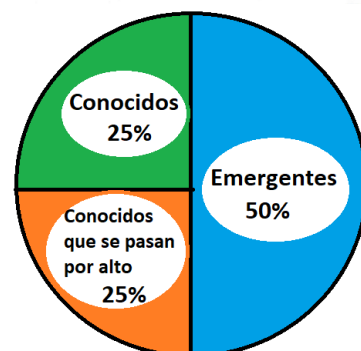
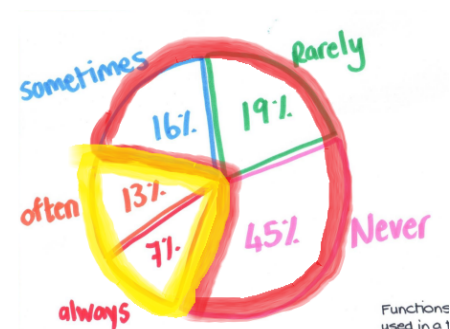
4. UVP, MVP, MMP, MLP - Mínimos Productos

El 80% del valor del producto está ubicado en el **20% de las características del producto**.

Para contrarrestar esto el agilismo plantea empezar a hacer **una versión del producto** sin tener todos los requerimientos.

Además del 100% de los requerimientos:

- Un 25% es conocido y se descubre
- Otro 25% es conocido pero se pasa por alto por error (acá juega la calidad del analista)
- Y un 50% de los requerimientos aparecen cuando el usuario se sienta a usar una versión del sistema. Estos requerimientos surgen cuando ya se está desarrollando el producto no importa cuánto se intente estos requerimientos no surgen hasta que el usuario no usa el sistema.



Para conseguir un producto sin desperdicio, y que logre cruzar la grieta, se recomienda iniciar con una versión del producto para después ir perfeccionándola.

Partimos de una **hipótesis** o un valor que tiene el producto que queremos desarrollar. Mi visión del producto va a ser poder resolver esa hipótesis única. Por ejemplo: el producto se va a vender el producto es viable, etc.

a) UVP - Producto Único Viable

Es el disparador, aquel producto único que quiero hacer. Tiene una hipótesis de valor único: El producto será único

b) MVP - Mínimo Producto Viable

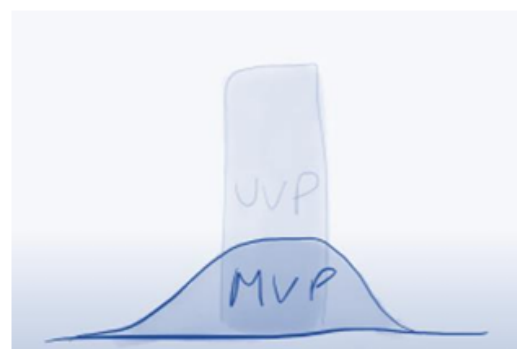
Es un producto limitado en funcionalidad construido para ver si se prueba la hipótesis.

La idea del MVP es trabajar **con lo mínimo del producto** que necesito obtener. Lo mínimo que tendría que tener el producto para que los (posibles) usuarios del mismo lo puedan evaluar y me puedan decir si el producto les atrae o no.

El objetivo del MVP no es vender el MVP sino **comprobar la hipótesis** del producto a construir, para ver si funciona o si tengo que hacer cambios. La idea es iniciar un circuito de aprendizaje a partir de la participación de los usuarios y la retroalimentación sobre si el producto cubre expectativas, sirve o si tengo que hacer cambios en las características incluidas.

i) MVP 2

Si con cada cliente potencial o con cada usuario que prueba la MVP recibo un feedback distinto, quiere decir que no está tan claro el mercado en el cual este producto va a funcionar) y significa que tengo que seguir trabajando sobre la visión del producto para encontrarlo.



También podría suceder que, en algún punto, la hipótesis planteada tenga algunas variaciones. Y la hipótesis puede moverse según la exigencia del mercado. Mi MVP **se va moviendo hacia el foco en el cual los clientes se van a interesar**. Si bien el foco elegido en la hipótesis no es erróneo, algunas características del mismo no están tan bien, por lo tanto, las desviamos.

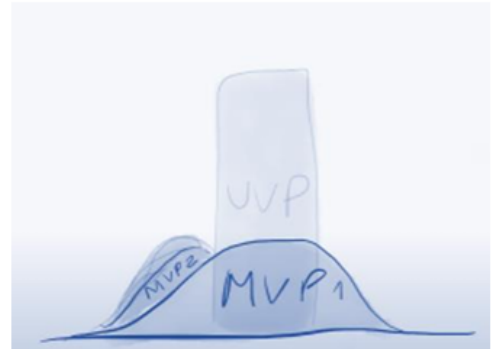


En este escenario debemos redefinir el MVP en base al conocimiento adquirido, por lo que creamos un nuevo MVP2. Este circuito de retroalimentación se vuelve a hacer, pero con este nuevo MVP.

No hace falta que el MVP sea código, podría ser por ejemplo un video

El MVP busca aquello que genera VALOR

Lo difícil es encontrar las pocas características que generen valor y poder probarlas con los clientes para validar la hipótesis.



Si limitamos el producto a una sola funcionalidad tenemos un **MVF (Minimal Viable Feature)**. Se trata de la característica mínima que se puede construir e implementar rápidamente, utilizando recursos mínimos, para probar la utilidad de la misma. Es una feature que buscamos que por sí misma ofrezca un valor agregado. La hipótesis se centra en una característica en vez de un producto.

Una vez que encontramos el MVP exitoso, es decir que el feedback y la investigación son exitosas, surge el concepto de:

c) MMP - Producto Mínimo Comercializable

Se enfoca en definir los elementos mínimos que deben estar presentes en un producto para que sea **comercializable**. Es la validación del producto en términos de la comercialización.

Ya no estoy parado en una hipótesis para validar si el producto tiene mercado o no, ya sé que sí lo tiene, ahora que estoy definiendo cual es la pieza mínima que voy a tener que construir y comercializar para que el producto sea rentable, y saber si va a funcionar la comercialización del producto antes de venderlo.

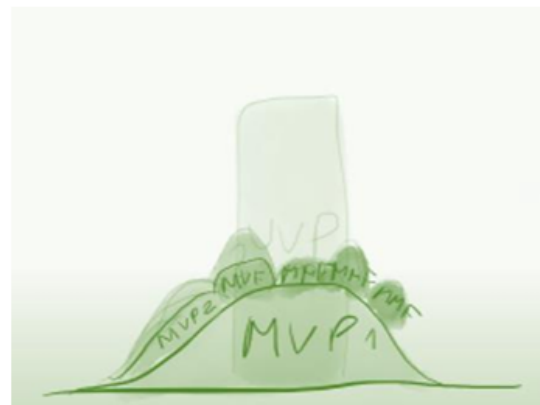


Es importante construirlo en cuestiones mínimas para **sacarlo rápidamente a la venta cuando el producto es atractivo y no está en el mercado**, ya que cuanto más tiempo pase más riesgo corres de que mi idea sea implementada antes.

Este sí debe ser código ejecutable que ande.

Si mandamos a producción a una sola funcionalidad tenemos un **MMF (Minimal Marketable Feature)**. Si la MMF resulta exitosa, se pueden desarrollar más MMF en esta área para tomar ventaja (ya que está comprobada). La razón para dividir una característica grande en MMF más pequeños es principalmente el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas.

Osea que podemos desarrollar un MVP o múltiples MMF.



d) MMR - Minimal Market Release

En el ciclo de vida de un producto, se van a sacar varias versiones del mismo. Cada versión que se saca al mercado es un release distinto, y cada release debe ser distinto al anterior.

El MMR es el release de un producto que tiene el conjunto de características más pequeño posible, EL incremento más pequeño que ofrece valor nuevo a los usuarios.

e) MLP - Producto Mínimo Adorable

Es la versión más simple de un producto que genera una conexión emocional o deleite en los usuarios. Enfatiza la satisfacción del usuario y la diferenciación.

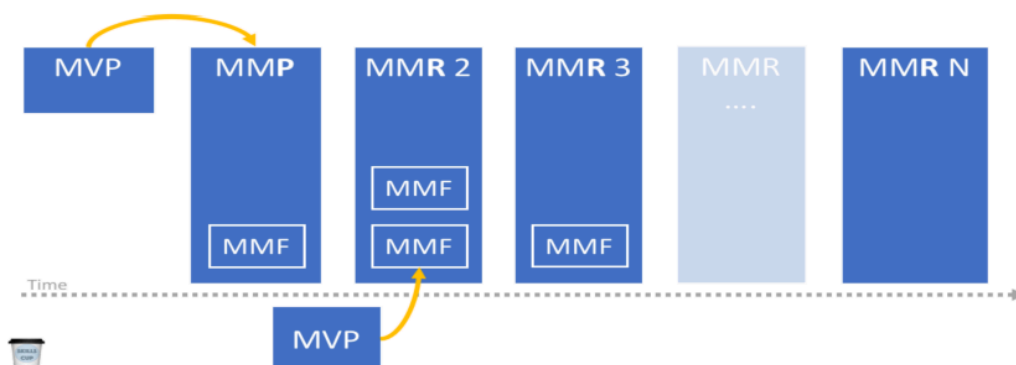
No elimina la hipótesis pero le da más valor a las características de diseño, usabilidad y fiabilidad.

Su objetivo es que el cliente NO rechace el producto.

¿Por qué hacer un MLP?

- **Reputación de marca:** No arriesgas la reputación poniendo el foco en tus usuarios.
- **Productos destacados:** Ayuda a las startups a destacar entre los usuarios de las grandes empresas.
- **Ciclo de retroalimentación del usuario:** Al captar la atención de los usuarios desde el principio, podrás capitalizar su interés a medida que el producto mínimo atractivo se desarrolla. Los usuarios ayudan porque confían en el producto.

5. Relación entre ellos



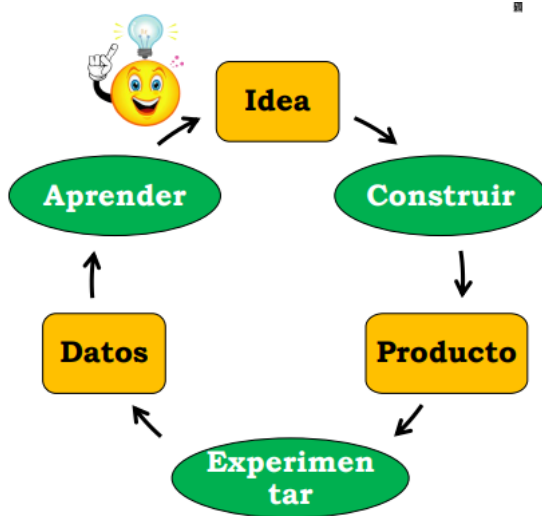
El MVP que compruebe la hipótesis permite armar el primer MMP, este MMP es el primer release osea MMP = MMR1. En cada release también se pueden incluir el MVP en forma de MMF o elegir ciertas MMF del MVP.

6. Valor VS Desperdicio

La idea de hacer un MVP es detectar qué produce valor para los clientes, es decir que necesitan. Esto busca reducir el desperdicio, osea no desarrollar funcionalidades innecesarias

Todo lo que no genera valor es desperdicio. Según lean hay que eliminar estos desperdicios para ser eficientes. Lo difícil es encontrar estas pocas características o features que realmente generen valor para poder probar una versión mínima del producto con el clientes y validar la hipótesis.

7. Construir- Experimentar- Aprender- Feedback loop

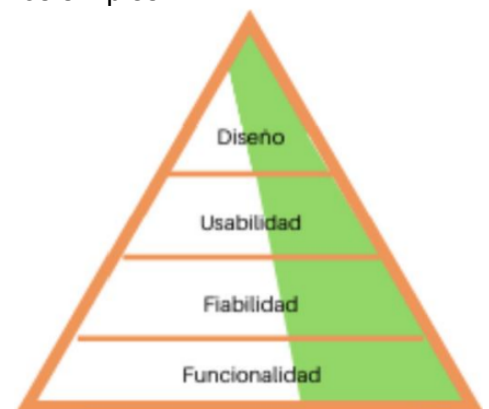


El éxito no es entregar un producto, **el éxito se trata de entregar un producto que el cliente usará.**

The Build-Experiment-Learn feedback Loop permite **descubrir las necesidades reales del cliente y alinear los esfuerzos continuamente hacia estas.**

8. Características de un MVP

Un MVP puede ir desde algo sumamente simple, poco más que un anuncio (smoke tests) hasta algo más complejo como un prototipos tempranos simples.



Cómo Sí hacer un MVP

A la hora de hacer un MVP, como dijimos antes, tenemos que elegir qué funcionalidades incluir y cuáles no, para esto usamos una matriz de priorización:

	Urgente	No urgente
Importante	1. Hacer Incluir en el MVP	2. Planear Desarrollar para un lanzamiento beta
No Importante	3. Delegar Considerar integraciones de terceras partes.	4. Eliminar Remover del Roadmap del producto

9. Dilema de audacia de cero - saltos de Fe

Cuando se tienen

- cero ingresos
- cero clientes
- cero tracción

se pierde el miedo a experimentar. Es por esto que tienes más probabilidades de éxito cuando tienes cero que cuando tienes algo que perder.

Ante esto hay que dar **saltos de Fe**, estos son los elementos más riesgosos de un plan, algunos son:

- **Hipótesis de valor:** Que el cliente quiera el producto
- **Hipótesis de crecimiento:** que al producto lo usen muchos clientes y que estos usuarios innovadores que suelen estar al tanto de las nuevas tecnologías se lo recomienden a otros usuarios.

10. Cómo preparar un MVP



PPT 6: Componentes de un proyecto de sw

Un proyecto es una unidad de gestión de trabajo. Es el medio por el cual gestionamos los recursos, tareas y personas para obtener un producto.

Como vimos el proyecto:

- Se ayuda de las definiciones del **proceso** para saber qué hacer, definen las tareas que voy a hacer eligiendo aquellas que sirvan.
- Tienen un **ciclo de vida** que indica cómo se van a llevar a cabo los procesos.

1. Características de un proyecto

- **Es de duración limitada:** son temporales, cuando se alcanza el/los objetivo/s, el proyecto termina. El proyecto tiene un inicio y un fin.
- **Es de elaboración gradual**
- **Tiene un resultado único e irrepetible:** Los proyectos tienen un objetivo único, es decir que cada resultado de un proyecto, es distinto del resultado de otro proyecto.
- **Tareas interrelacionadas:** Las tareas dependen una de otra, tienen precedencia y a su vez tienen asociados/designados recursos y esfuerzos.
- **Orientados a objetivos:** su resultado esperado está fijado en su objetivo. Estos guían al proyecto y deben ser claros y alcanzables.

2. Administración de proyectos

Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto.

Incluye

3. La triple restricción

Hay que recordar que en un proyecto vamos a tener una restricción de tiempo, de presupuesto y un alcance, El PM debe balancear los 3 objetivos que están involucrados en el proyecto.

4. Plan de proyecto

Es como una hoja de ruta. Cuando nos sentamos frente a un proyecto debemos **delinear un plan de proyecto** que va a esbozar **qué es lo que va a hacer el proyecto**:

A través del plan documentamos:

- ¿QUÉ hacemos? → Alcance
- ¿CUÁNDO lo hacemos? → Calendarización
- ¿CÓMO LO HACEMOS? → Recursos y decisiones disponibles
- ¿QUIÉN LO VA A HACER? → Asignar tareas

La idea del plan de proyecto es que se vaya nutriendo y corrigiendo a medida que el proyecto se lleva a cabo, hay que actualizarlo permanentemente.

Para armar este plan del proyecto debemos **planificar**:

5. Planificación de proyectos de software - Con procesos definidos (Enfoque tradicional)

¿QUÉ VAMOS A HACER?

- 1) **Definir el objetivo:** Construir un producto de scon todo lo que está en la ERS.
- 2) **Definir el alcance:** Es todo el trabajo y solo el trabajo que debe hacerse para cumplir con el objetivo. Son **actividades**. Se mide contra el **Plan del proyecto**.

¿CÓMO LO VAMOS A HACER?

- 3) **Definir el proceso y el ciclo de vida del proyecto:** Según mi equipo, el cliente, y qué producto voy a hacer, defino los procesos que me sirven para este proyecto.
 - La idea es que el proceso describa con detalle **qué hacer** y en base a eso nosotros decidimos qué **tareas llevar a cabo y cuáles no**, justificando la elección.
 - El ciclo de vida es **de qué manera** ejecutó esas actividades que elegí.

¿QUIÉNES LO VAN A HACER?

- 4) **Definir el equipo del proyecto:**

En el enfoque tradicional el líder es quien toma las decisiones.

Cada persona del equipo de desarrollo tendrá un **ROL**.

- Qué roles habrá en el proyecto.
- Las responsabilidades de cada rol las indica el proceso. Para tener flexibilidad los procesos trabajan con roles chicos y muy específicos, puede que una persona tome varios roles.

- 5) **Estimación (ya lo expliqué)**

- 6) **Calendarización /Programación del proyecto**

Se anticipa con mucho detalle qué tareas se van a hacer y quienes las van a hacer.

Tipos de asignaciones:

- Asignación anticipada o PUSH → El líder del proyecto asigna las tareas
- Asignación diferida o PULL → La asignación se hace en base al proyecto y sus tiempos.

- 7) **Gestión de riesgos:** Se listan los riesgos más probables de ocurrir o que tengan mayor impacto en el sistema. Multiplicando estas dos variables obtenemos la **exposición de riesgo**. Según esto decido qué riesgos son principales y cuáles gestionar.

EXPOSICIÓN DE RIESGO = PROBABILIDAD DE OCURRENCIA X IMPACTO EN EL SISTEMA

Una vez identificados los gestionamos, osea generamos acciones para disminuir el impacto o evitar la ocurrencia.

- 8) **Planificar el seguimiento y control de proyecto:** Esta actividad se planifica, cada cuanto se va a hacer, qué informes necesita, etc.



Para llevarse a cabo necesita:

9) **Métricas:** Estos son números objetivos. Nos van a permitir darnos cuenta si nuestro proyecto está en línea con lo planificado o si está desviado. Comparamos lo planificado con lo real.

- a) Métricas de proceso: Nos permiten saber si la organización está trabajando bien o mal. Por ej: porcentaje de proyectos en término y porcentaje de proyectos fuera de término.
- b) Métricas de proyecto: Me van a permitir saber si un proyecto de software en ejecución se está cumpliendo de acuerdo a lo planificado o no. Por ej:
Tiempo planificado contra tiempo real.
 - i) Tamaño
 - ii) Esfuerzo
 - iii) Tiempo
 - iv) Defectos
- c) Métricas de producto: Miden cuestiones/características que tienen una relación directa con el producto que estamos construyendo. Ej: Métricas de defectos, métricas de tamaño, etc.

Monitoreo y control

El PM se va a encargar de comparar lo definido en el plan de proyecto y lo real para determinar si se está cumpliendo o no. Para hacer esto se va a basar en el plan de proyecto y en las métricas.

Alcance del producto vs alcance del proyecto

El producto tiene:	El proyecto tiene
• Objetivo: Para qué armo el sistema. Por ej: Darle soporte a x tarea. • Alcance: Son los requerimientos funcionales y no funcionales necesarios para cumplir con el objetivo • Aquello que no se contempla: lo que está fuera del alcance TODO ESTO SE GUARDA EN LA ERS	• Objetivo: Cumplir con lo que está en la ERS. • Alcance: El trabajo y solo el trabajo para cumplir con su objetivo

Ciclo de vida del producto y ciclo de vida del proyecto

El **ciclo de vida del producto** dura desde la concepción de su idea hasta que este se retira del mercado. Durante el ciclo de vida de un producto este va teniendo distintas versiones, cada una diferente a la anterior.

El **ciclo de vida del proceso** dura lo que dura el desarrollo de un release.

El **ciclo de vida del producto** está constituido por muchos ciclos de vida de proyecto.



Ppt 7: SCRUM

Scrum es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

Resumidamente:

Scrum necesita a un **Scrum master** para que fomente un ambiente donde:

1. Un **Product Owner** ordene el trabajo de un problema complejo en un **Product Backlog**
2. El equipo de desarrollo convierta una sección del trabajo en un incremento de valor durante el **sprint**.
3. El **equipo de desarrollo** y los **stakeholders inspeccionen** los resultados y **ajusten** para el próximo sprint
4. **Repetir**

1. Teoría de Scrum

Scrum tiene sus fundamentos en el empirismo y el pensamiento lean. Se apoya en los pilares de transparencia, inspección y adaptación.

Los pilares del empirismo en scrum:

- **Transparencia:** El trabajo debe ser visible para los que estén haciendo el trabajo y aquellos que lo reciban. Sin eos se pueden tomar decisiones desinformadas que aumenten el riesgo y disminuyan el valor.

La transparencia permite la:

- **Inspección:** Hay que inspeccionar continuamente los artefactos y el progreso a los objetivos para detectar problemas.
- **Adaptación:** Si algún aspecto del proceso se sale de los límites aceptables o si el producto resultante es inaceptable hay que ajustar el proceso aplicado o los materiales producidos lo antes posible.

2. Valores de scrum

Compromiso, Foco, Apertura, Respeto y Coraje → FACCRR

3. Scrum Team - Equipo Scrum

La unidad fundamental de Scrum es un pequeño equipo de personas.

- En cada equipo no hay sub-equipos o jerarquías.
- Los equipos son multifuncionales, sus miembros reúnen todas las habilidades necesarias para crear valor en cada sprint.
- Son auto gestionados, es decir que deciden quién va a hacer qué y cuándo de manera interna
- Los equipos deben tener una cantidad de personas que les permita ser ágiles pero a la vez completar cantidades significativas de trabajo.
- Los equipos deben estar enfocados en el mismo producto, es decir que deben tener el mismo:
 - Objetivo de producto
 - Product Backlog
 - Product Owner

Miembros del Scrum Team

- **Desarrolladores:** Cualquier miembro del equipo encargado de crear cualquier aspecto de un incremento en un sprint. Sus tareas incluyen:
 - Crear el plan para el sprint y el Sprint Backlog
 - Instalar calidad adhiriendo a la Definition of Done
 - Adaptar sus planes cada día en dirección al objetivo del sprint (Sprint goal)
 - Responsabilizarse entre sí profesionalmente
- **Product Owner:** Debe maximizar el valor del producto resultante. También es responsable de un manejo efectivo del Product Backlog.
 - Desarrollar y comunicar el objetivo del producto.
 - Crear y comunicar los items del Product Backlog
 - Ordenar los items del Product Backlog.
 - Asegurar que el Product Backlog es transparente, visible y entendido.

El PO es UNA SOLA PERSONA, puede delegar responsabilidades (Proxy PO) y representar a varios stakeholders pero sigue siendo el responsable.
Si alguien quiere cambiar el product backlog debe convencer al PO.
- **Scrum Master:** Debe establecer Scrum como está definido en la guía. Ayudan a que todos entiendan Scrum dentro del equipo y de la organización. También es responsable por la efectividad del equipo. **Para esto este sirve al equipo:**
 - Entrenando a los miembros del equipo en autogestión y multifuncionalidad.
 - Ayudando al equipo a crear incrementos de alto valor
 - Ayudando a remover impedimentos al progreso del equipo
 - Asegurando que todos los eventos se hagan, sean positivos y mantenidos dentro de los tiempos estipulados.

También **sirve al Product Owner** y a **la organización**.

4. Eventos de Scrum

Cada evento es una oportunidad para **inspeccionar y adaptar** los artefactos de Scrum. Estos eventos están diseñados para permitir la **transparencia** necesaria.



Sprint

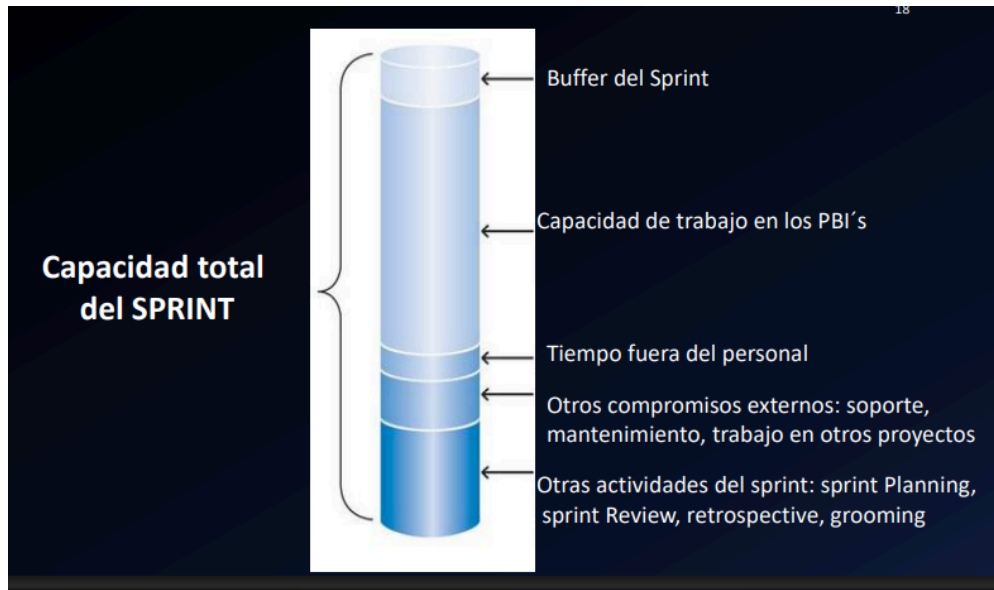
Eventos de duración fija en los que se hace todo lo necesario para cumplir el objetivo del producto. Un sprint empieza después del otro automáticamente.

- No se hacen cambios que arriesguen el objetivo del sprint
- La calidad no decrece
- Se redefine el product backlog en tanto se necesite
- Se puede revisar y renegociar el alcance con el PO

Un sprint podría ser cancelado si el objetivo del sprint se convierte en obsoleto.

TIMEBOX: Máximo 1 mes

Dentro de un sprint no se usa el 100% del tiempo para avanzar en el producto:



Sprint Planning

Se hace al inicio del sprint por todo el equipo y por invitados que puedan aconsejar. El PO se asegura de que todos conozcan los ítems más importantes del Product Backlog. Acá se define el INCREMENTO del producto.

Su objetivo es obtener el Sprint Backlog

Temas que trata:

- **¿Por qué es valioso el sprint?:** → El PO describe cómo incrementar el valor del producto, el equipo define un sprint goal que comunique ese valor.
- **¿Qué se puede hacer en el sprint?** → Se seleccionan ítems del Product Backlog para incluirlos en el sprint actual.
- **¿Cómo se va a hacer el trabajo elegido?** → Por cada ítem los desarrolladores planifican el trabajo necesario para generar un incremento que cumpla con el Definition of done

El sprint goal + los ítems del Product backlog + La planificación = SPRINT BACKLOG

TIMEBOX: Máximo 8 horas para un Sprint de un mes

Acá se estiman los SP a los que se va a comprometer el equipo en base a el objetivo del sprint, la duración, la configuración del equipo y su capacidad.

Daily Scrum

Inspeccionar el progreso hacia el sprint goal y ajustar el sprint backlog. Lo hacen los desarrolladores, si el PO y el scrum master están trabajando en el Scrum Backlog también participan como desarrolladores.

Acá se actualiza el board del sprint.

TIMEBOX: 15 minutos

Daily Review

El equipo presenta los resultados de su trabajo a los stakeholders.

El equipo y los stakeholders evalúan revisan que se logró, qué cambio, y definen qué se hará después y ajustan el Product Backlog de ser necesario.

TIMEBOX: 4 horas para un Sprint de un mes

Sprint Retrospective

El equipo revisa cómo fué el último sprint, que salió bien, qué problemas tuvieron, y cómo esos problemas se resolvieron (o no).

Lo primero que se aborda es lo que más impacto tiene.

A partir del segundo sprint podemos usar la **velocidad** como métrica.

TIMEBOX: 3 horas para un Sprint de un mes

5. Artefactos de Scrum

Representan trabajo o valor. Están diseñados para maximizar la transparencia.

A qué le da transparencia cada artefacto

1. Product backlog → Product Goal
2. Sprint Backlog → Sprint Goal
3. Incremento → Definition of Done

Product backlog → PRODUCTO

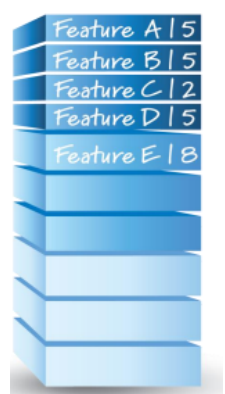
Es una lista ordenada de todo lo que hay que mejorar en el producto. Estos son los Product Backlog Items (PBI).

Además de US este puede tener: Spikes, restricciones, RNF (tech stories), Épicas, Temas, deuda técnica, defectos (no en la configuración inicial).

Básicamente **cualquier manera de describir características que el producto debe tener.**

- Para iniciar el producto necesito la cantidad de ítems necesarios para hacer el 1er sprint.
- Se puede refinar el Product backlog para obtener ítems más pequeños y precisos y mantener al backlog organizado y consistente, en cualquier momento del sprint.

Product backlog



TIMEBOX de refinamiento: 10% del tiempo del sprint

Product Goal

Describe el estado futuro del producto. Está en el product backlog junto con qué va a cumplir este objetivo.

Los ítems del product backlog que pasen la **Definition of Ready** pueden pasar al **Sprint backlog**.

Sprint backlog → TAREAS

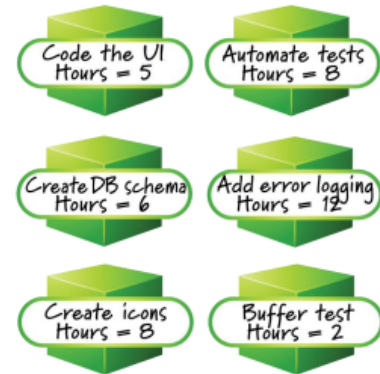
Compuesto por:

- Sprint Goal (porqué)
- Ítems del product backlog seleccionados para el sprint (qué)
- Plan accionable para entregar el incremento (cómo)

Es una imagen del **trabajo** que los desarrolladores planean llevar a cabo para cumplir con el Objetivo del sprint.

Es actualizado durante el sprint

Sprint backlog tasks



Sprint Goal

Es el objetivo único del sprint

Para un sprint backlog tengo un TaskBoard

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... SC Test the... SC Test the... SC Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC Test the... SC Test the... SC 6
	Code the... 4	Code the... 6			

Historias que pasaron el
criterio de ready
Definidas en el SPRINT
PLANNING

Estimadas en Story
Points

Tareas necesarias para
completar las US

Estimadas en esfuerzo: horas de
trabajo productivas

Incremento

Es el resultado del sprint que se presenta al PO. Cada incremento es aditivo al anterior, todos trabajan en conjunto. Puede haber muchos incrementos en un solo sprint. Cualquier trabajo hecho NO puede ser considerado parte de un incremento a menos que cumpla con la **Definition of Done**.

Definition of Done

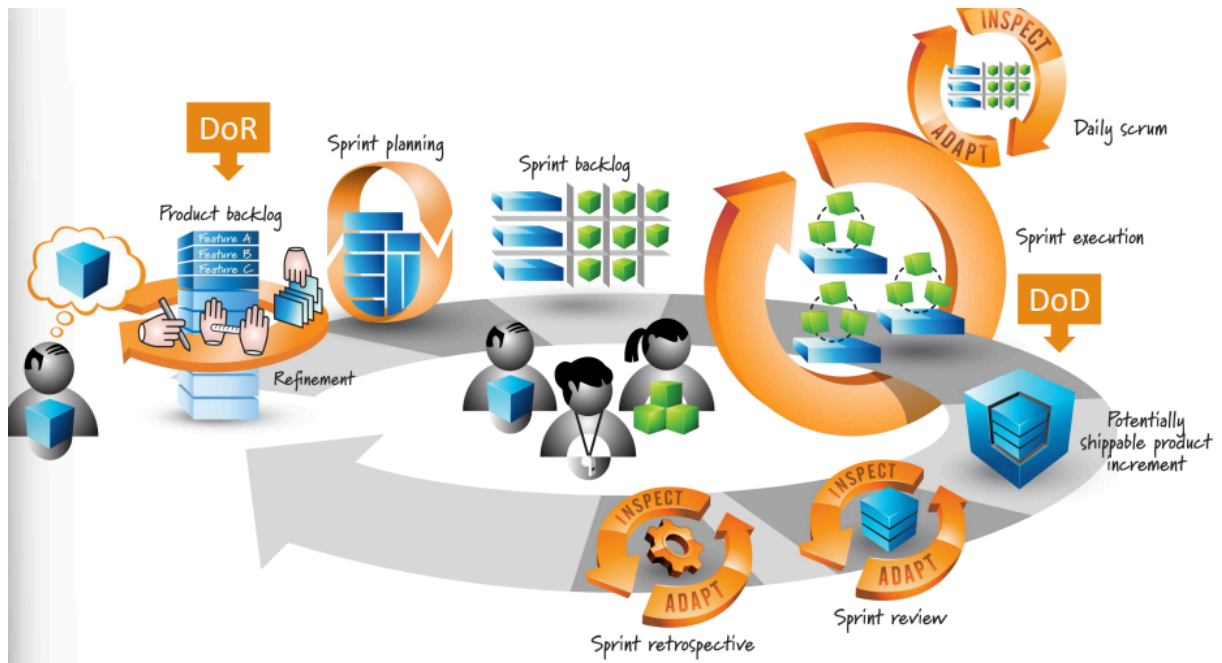
Descripción formal del estado que debe tener un incremento para cumplir con las medidas de calidad requeridas para el producto.

Cuando un ítem del product backlog pasa la definición de Done se crea un incremento.

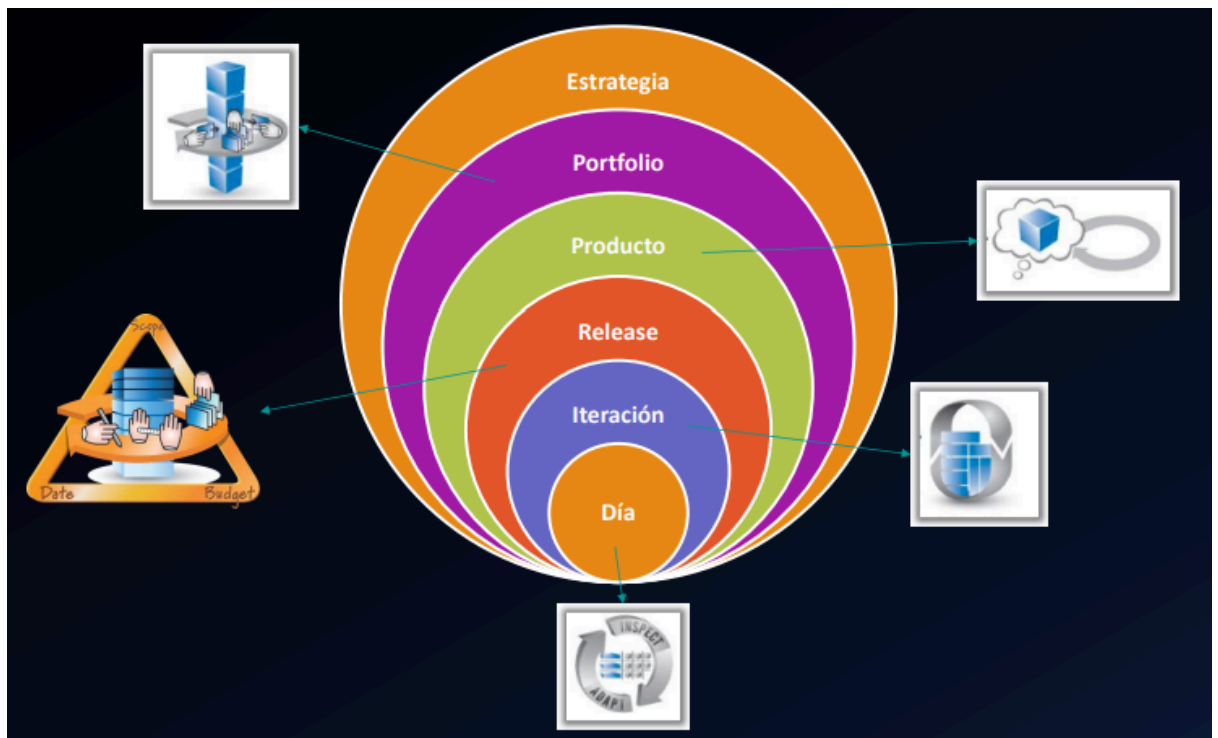
Velocidad:

Es una métrica que mide producto según la cantidad de software funcionando que el PO me aceptó. Es la suma de story points. NO ES UNA ESTIMACIÓN.

Manifiesto ágil: La mejor métrica de progreso es software funcionando.



6. Niveles de planificación

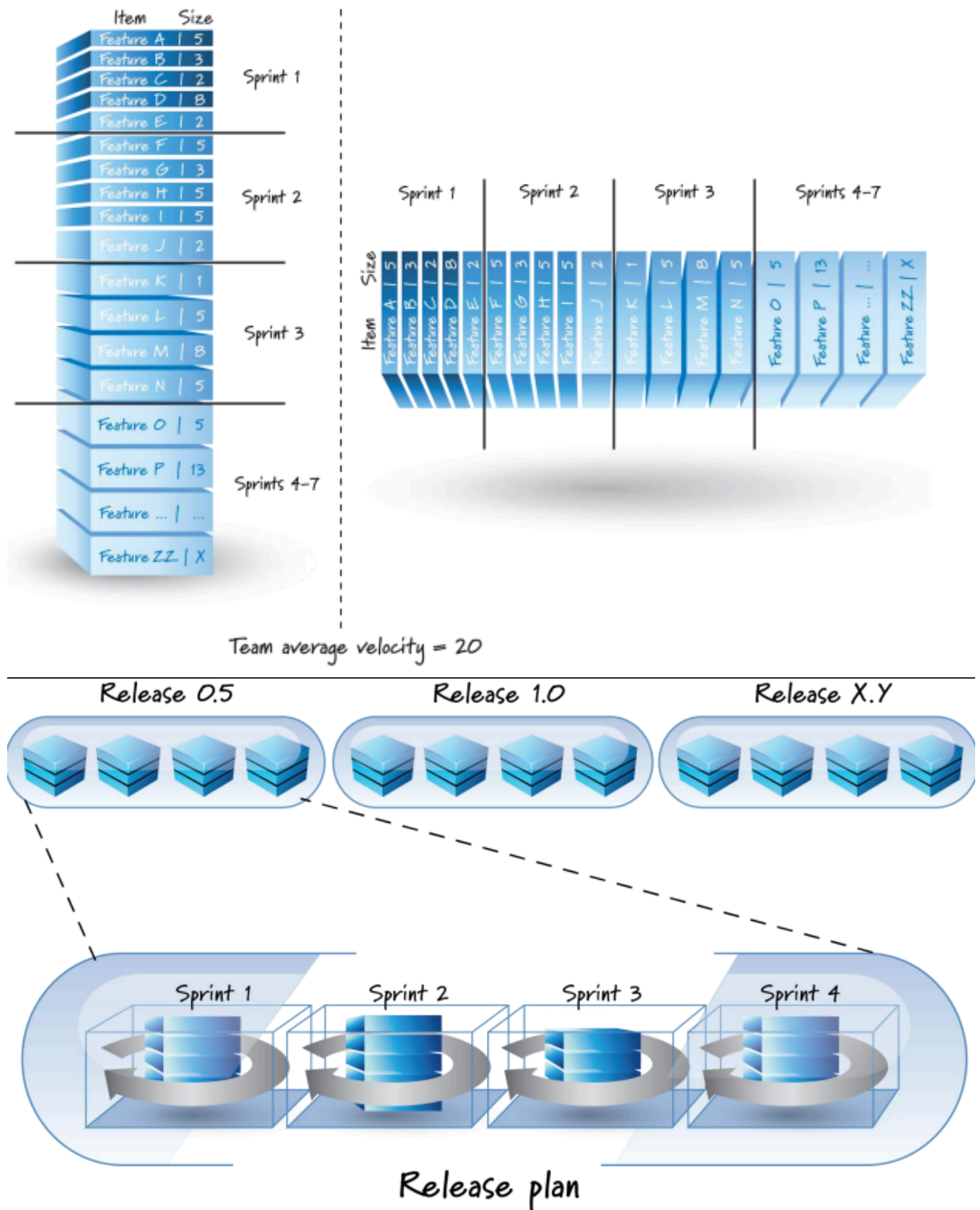


Scrum funciona a nivel de día (daily) e iteración (sprint)

Planificación de release → Cuánto dura y cuál es el incremento de producto que quiero

Por encima de la planificación del sprint está la planificación del RELEASE o incremento del PRODUCTO. Una versión del producto.

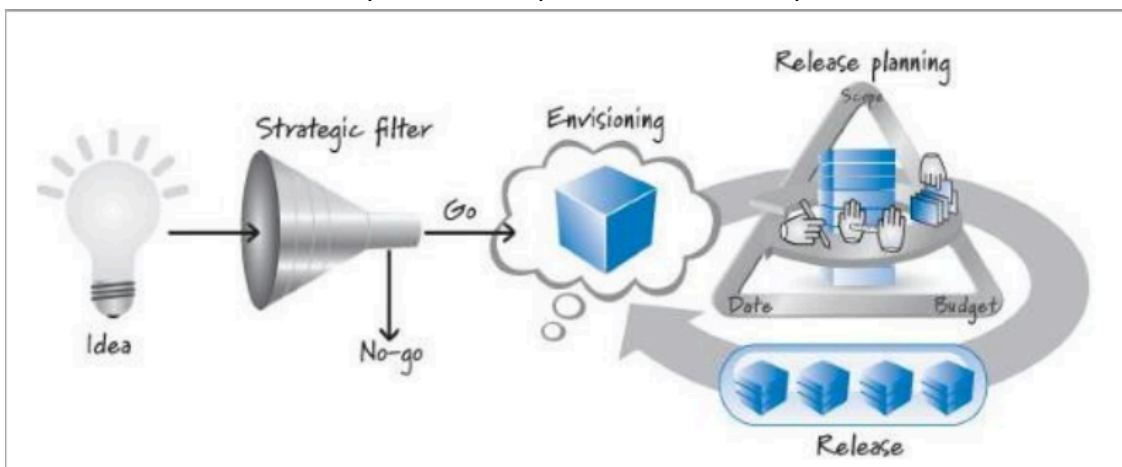
Por lo general el release se hace luego de múltiples sprints pero esto puede variar.





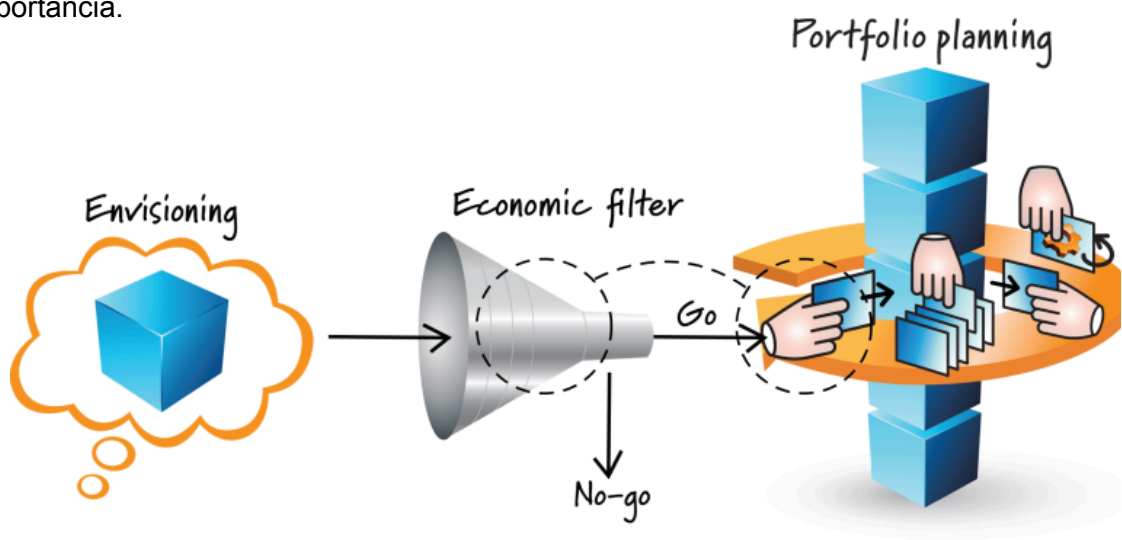
Planificación de producto

Por encima del release está el producto, un producto tiene múltiples release.



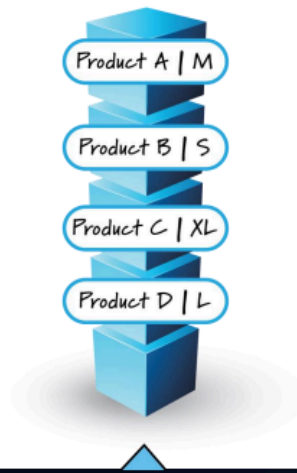
Planificación de portfolio

Está por encima de la planificación del producto. Acá se define a qué producto se le da más importancia.



¿Qué y Cuándo estimar?

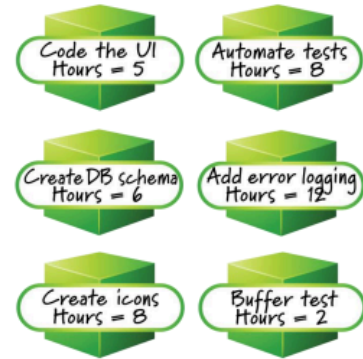
Portfolio backlog



Product backlog



Sprint backlog tasks



No Silver Bullet

El artículo afirma que no hay un solo avance, ni en tecnología ni en técnicas de gestión, que puedan en sí mismas prometer una mejora mínima en la productividad, confianza y simplicidad en el desarrollo de software. Para explicar porque, examina la naturaleza de las propiedades del software y las “silver bullets” o soluciones milagrosas propuestas para lograr esto.

Las silver bullets son propuestas que supuestamente van a resolver los problemas de costo, atrasos, presupuestos y productos fallidos que pueden surgir del desarrollo de un sistema.

Para ver que tanto progreso se puede esperar de la tecnología de software se examinan las dificultades “esenciales” o inherentes al software y las dificultades “accidentales” del mismo.

Dificultades esenciales:

El software es en esencia abstracto, aún así es altamente preciso y extremadamente detallado. Según el autor, la parte difícil de hacer software no es el trabajo de representar el sistema y testear su representación, sino más bien especificarlo, diseñar y testear la construcción del concepto del mismo.

Si esto es verdad, construir software siempre va a ser difícil. No hay una “silver bullet”.

Propiedades inherentes al software:

- 1) **Complejidad:** El software es complejo, porque no tiene dos partes iguales, ni siquiera similares. Si encontramos dos partes parecidas las convertimos en una, así cada aparte de un sistema es algo completamente diferente a la otra.
Incluso la mejora o crecimiento de un sistema no es en lo más mínimo una repetición de los mismos elementos pero más grandes, sino un aumento en el número de distintos elementos diferentes entre sí.
Además, en la mayoría de los casos los elementos interactúan de una manera no lineal, lo que hace que la complejidad aumente de una forma mucho más que lineal. Una descripción de software que remueva su complejidad, removerá su esencia.
De esta complejidad viene la dificultad de la comunicación entre los miembros del equipo, lo que lleva a fallas de producto, sobrecostos(?), y atrasos en el cronograma. De esta misma complejidad viene la dificultad de nombrar y mucho menos entender los posibles estados de un programa y de ahí viene la falta de confiabilidad en el mismo. No solo se ocasionan problemas técnicos por esta complejidad, sino también problemas de gestión, la complejidad hace difícil la revisión, y crea una gran carga de aprendizaje y comprensión.
- 2) **Conformidad:** Muchas veces el ingeniero de software tiene que superar complejidades arbitrarias, forzadas sin razón por humanos, instituciones y sistemas. Estas difieren de interfaz a interfaz, y de tiempo en tiempo, no por necesidad, sino porque fueron diseñadas por distintas personas. En muchos casos un software se tiene que acomodar o conformar porque es el que fue desarrollado más recientemente o porque es el que mejor se podría conformar o adaptar. En todos los casos, mucha complejidad viene de conformar a otras interfaces.

- 3) **Posibilidad de cambiar:** El software está continuamente sujeto a presiones de cambio. En productos de manufactura los cambios se introducen en nueva versiones o modelos, pero raramente sobre un mismo ítem o producto.
- Por otro lado, el software en un sistema encarna su funcionalidad, y la funcionalidad es la parte más subjetiva a cambio. Todo software exitoso se cambia, a medida que un producto de software es más conocido por ser de utilidad más personas lo prueban al borde, o directamente fuera de su dominio inicial. Además el software exitoso sobrevive más que la vida útil de las máquinas para las que fue escrito. Si no son nuevas computadoras serán nuevos discos, displays, impresoras, etc; que vendrán y el software deberá ser acomodado a sus nuevos vehículos de oportunidad.
- 4) **Invisibilidad:** El software es invisible. La realidad del software no está embebida en el espacio, no tiene representación geométrica. Eso priva a la mente de una de sus herramientas conceptuales más importantes. Esta carencia no sólo impide el proceso de diseño en la mente, también perjudica la comunicación entre mentes.

Breakthroughs que resolvieron dificultades accidentales

- 1) **Lenguajes de alto nivel:** El ataque más poderoso a favor de la productividad, confiabilidad y simplicidad del software fue el uso progresivo de los lenguajes de alto nivel para programar. Estos lenguajes liberan al programa de muchas de sus dificultades accidentales. Un programa abstracto consiste en construcciones conceptuales. La máquina concreta donde corre el programa se enfoca en bits, registros, condiciones, ramas, canales, etc. El lenguaje de alto nivel elimina todo un nivel de complejidad que nunca fue inherente al programa. Sin embargo, en cierto punto la elaboración de un lenguaje de alto nivel se convierte en una carga que incrementa, no reduce, la tarea intelectual de su usuario.
- 2) **Time-sharing:** Time-sharing preserva intermediación, y por lo tanto nos permite mantener una visión general de la complejidad. La lentitud de la programación por lotes implica que nos olvidaremos los detalles minuciosos o incluso lo que estábamos pensando antes de dejar de programar y ser llamados a compilar y ejecutar el programa. Esta interrupción es costosa en tiempo, su efecto más serio es probablemente la disminución de entendimiento en todo lo que está sucediendo en un sistema complejo. La lentitud en la entrega, al igual que las complejidades del lenguaje de máquina, es una dificultad accidental más que esencial del proceso de software.
- 3) **Entornos de programación unificados:** Los entornos de programación integrados atacan las dificultades accidentales de usar varios programas juntos, proveyendo de librerías integradas, formatos de archivos unificados y montones de filtros. Este avance estimuló el desarrollo de bancos de herramientas completos, ya que cada herramienta se podía aplicar a cualquier programa usando formatos estándar.

Esperanzas de una silver bullet:

Desarrollos técnicos que a menudo se presentan como posibles soluciones milagrosas:

- **Ada y otros lenguajes de alto nivel**
- **Programación orientada a objetos**
- **Inteligencia artificial**
- **Sistemas expertos**

- Programación “automática”
- Programación gráfica:
- Verificación de programa:
- Entornos y herramientas:
- Workstations (estaciones de trabajo):

Ataques prometedores a la esencia conceptual

- Comprar vs construir:
- Refinamiento de requerimientos y prototipado rápido
- Desarrollo incremental: desarrollar, no construir, software
- Buenos diseñadores

CONCEPTO	ANCLAJES
Software	• Compleja, intangible. • Artefactos, desarrollo. • Código, documentación, herramientas
Software de aplicación	• Tareas • Necesidades
Ingeniería de software	• disciplina • producción (del sw)